

Data Structures and Algorithms

- Lini Thomas + kshiti Gajjar +
Venkatesh Chopella

~~Sorting~~

~~Sorting~~

~~Sorting~~

~~Sorting~~

~~Sorting~~ 03/01/25

~~Sorting~~

→ Sorting:

1) insertion sort: make room for the new element, then insert it

i.e. elements get shifted one-by-one till we find the proper position for required element

~~pseudocode~~ ~~for (i = 1; i < n; i++) {~~

~~for (j = i - 1; j > 0; j--) {~~

~~if (arr[i] < arr[j]) {~~

~~arr[j + 1] = arr[j];~~

~~arr[j + 1] = arr[i];~~

~~arr[j] = arr[j + 1];~~

~~arr[j] = arr[j + 1];~~

~~}~~

~~}~~

int arr[n]

int temp;

pseudocode: for (i = 1; i < n; i++) {
for (j = i - 1; j > 0; j--) { temp = arr[i];

if (temp < arr[j]) {

arr[j + 1] = arr[j];

} else {

arr[j + 1] = temp;

}

}

}

complexity: $O(n^2)$ as in worst case, we are running one n-sized loop, and for every element in that n-sized loop, we run another n-sized loop ~~for (j = i - 1; j > 0; j--)~~

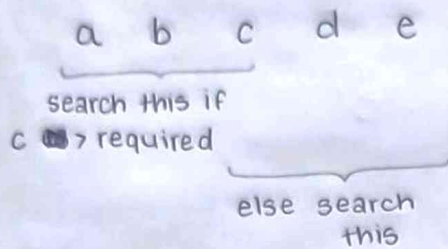
It is $O(n)$ in the best case i.e. an already sorted array, as the second n-sized loop doesn't need to run at all.

★ H.W:

1. Code Insertion Sort
2. Code Binary Search
3. Bubble Sort
4. Selection Sort

• Binary Search

→ jump to the middle element of a sorted array, check if that number is less than or greater than required element:



∴ number of steps halved each time.

i.e. complexity = $O(\log_2 n)$

↳ because $1/2$

→ This is better than scanning the array fully, which is an $O(n)$ process.

→ Q. Can Binary Search help us with insertion sorting?

A. No, even though ~~finding~~ finding the location will be faster, we still have to shift the element there.

i.e. insertion sort using binary search

$O(n \times n \log n)$

↓ finding
shifting position ×
+ n values
placing of i-loop

Q. An inversion = if $i < j$ and $a[i] > a[j]$, $(a[i], a[j]) \rightarrow$ Inversion.

If there are $f(n)$ inversions, what is the relation between inversions and insertion sort?

A. number of steps in an insertion sort = number of inversions in the permutation.

At any point, we only need to focus on the inversions of the number that lie before the number, as the inversions after the number are covered by the numbers after it

||

we check for all the numbers greater than current i value (to shift it).

∴ checking for inversions = checking for insertion sort.

∴ number is the same.

ii) Bubble sort : swapping consecutive pairs of elements from the first pair (0^{th} - 1^{st} element), moving till the $(n-1)^{\text{th}}$ position in each iteration i.e. there are $n-1$ iterations.

Best case complexity is $O(n)$, where the array is already sorted, and further ~~more~~ iterations can be stopped.

iii) Selection sort : select the maximum number, put it in the last place of the current iteration : $(n-1)^{\text{th}}$ position, i.e. iteration number, and placing the number currently at last position to the original position of the maximum.

Cor. find smallest & put in first position) contains minimum swaps.

Q. Let A be an array containing integer values. $\text{dist}(A)$ = minimum no. of swaps required to sort array.

$\text{dist}(2, 5, 3, 1, 4, 2, 6)$

A. $\begin{array}{ccccccc} 2 & 5 & 3 & 1 & 4 & 2 & 6 \\ & \uparrow & & \uparrow & & & \\ 2 & 2 & 3 & 1 & 4 & 5 & 6 \\ & \uparrow & & \uparrow & & & \\ 2 & 2 & 1 & 3 & 4 & 5 & 6 \\ & \uparrow & & \uparrow & & & \\ 2 & 1 & 2 & 3 & 4 & 5 & 6 \\ & \uparrow & & \uparrow & & & \\ 1 & 2 & 2 & 3 & 4 & 5 & 6 \end{array} = 4.$

Q. Which sort uses least amt. of comparisons to sort:

23 32 45 69 72 73 89 97

A. Insertion or Bubble sort

Q. number of comparisons to find an element in a list of n (distinct) elements that is neither minimum nor maximum?

A. Look at first 3 elements, pick the middle valued one.

Q. Best Algorithm to Find number of leaders (numbers that are greater than all numbers to its right).

A. Move ~~from~~ from right to left, find no. of ^{new} maximums.

Q. Best algo to shift all negative elements to the left & all positive elements to the right contains how many iterations?

07/01/25

→ Merge Sort: merging two sorted lists into one (divide and conquer algorithm)
 ↳ minimum time complexity of any sorting algo: $n \log n$

Ex:

99	6	86	15	58	35	86	4	0
----	---	----	----	----	----	----	---	---

99	6	86	15
----	---	----	----

58	35	86	4	0
----	----	----	---	---

99	6
----	---

86	15
----	----

58	35
----	----

86	4	0
----	---	---

99	6	86	15
----	---	----	----

58	35	86	4	0
----	----	----	---	---

99	6	86	15	58	35	86	4	0
----	---	----	----	----	----	----	---	---

→ after getting multiple 1-element (i.e. sorted) arrays,
 we can merge sort them to get the final sorted array.

Q. 56, 5, 7, 2, 99, 17, 3, 9, 85

A. 56, 5, 7, 2 99, 17, 3, 9, 85

56, 5 7, 2 99, 17 3, 9, 85

56 5 7 2 99 17 3 9, 85

56 5 7 2 99 17 3 9 85

5, 56 2, 7 17, 99 3, 9, 85

2, 5, 7, 56 3, 9, 17, 85, 99

2, 3, 5, 7, 9, 17, 56, 85, 99

2, 3, 5, 7, 9, 17, 56, 85, 99

unsorted array

split

obtained 1-element arrays

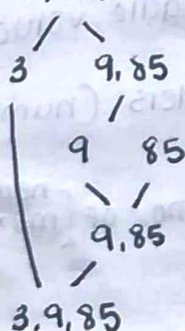
merge
(while sorting)

sorted array

both arrays
done
simultaneously
technically
incorrect
flow

we merge back what we had split.

i.e. 3, 9, 85



→ recursive algorithm: for every new array created due to splitting, merge sort is called on that new array.

i.e.:

a	b	c	d
---	---	---	---

a	b
---	---

c	d
---	---

1. merge sort this

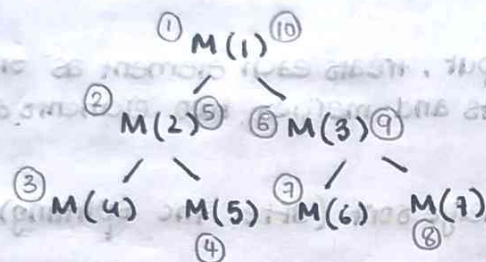
2. merge sort this

3. merge the sorted halves together

★ Two-way merge efficiency compared to merge sort?
 • prove that merge sort by splitting is more costly > 2

merge sort → splitting of array → merge sort of new arrays
 ↗ recursion ↖

→ flow/sequence of recursion:



→ while merge-sorting, we can either create a temporary array for each step, which takes up a lot of space. ~~or~~ or we can use the same array repeatedly

i.e. take array of size n , and fill its positions with the appropriate values.

this is called double storage merging.

→ complexity of merge-sort algorithm: there is no difference in no. of steps b/w the best and worst case $\therefore$ complexity is $n \log n$ in any case.

→ Time to sort n elements = $T(n)$

$$T(n) = T(n/2) + T(n/2) + c \cdot n$$

Time to sort first half

Time to sort second half

→ some time

to merge the sorted halves. merging is $O(n)$ process.

$$\hookrightarrow T(n/4) + T(n/4) + c \cdot n/2$$

⋮

$$T(n) = 2^k T(n/2^k) + k \cdot c \cdot n \text{ in the } k^{\text{th}} \text{ step}$$

we know the process goes on until n cannot be divided anymore

$$\therefore n/2^k \leq 1$$

$$n \leq 2^k$$

$$k \geq \log_2 n$$

$\therefore k = \log_2 n$ where array all have only one element

$$(i.e. \ T(1), \ T(n/2^k))$$

$$n/2^k = 1$$

$$\therefore T(n) = 2^{\log_2 n} T(n/2^{\log_2 n}) + c \log_2 n \cdot n$$

$$T(n) = n T(1) + cn \log_2 n$$

$$\therefore T(n) = n \log n \quad (c=1, \text{ let})$$

→ two-way merge: takes array as input, treats each element as single element array, and sorts and merges two elements at a time

i.e. bottom → top of merge sort (after the splitting)

Q. What if we split into > 2 parts?

A. It is possible to sort the array this way, and the no. of steps will be less, but there are more complex operations (comparison) etc. that are now required.

Thus, it is more costly.

→ Binary vs ternary search

$$T_B(n) = T(n/2) + 2c$$

$$T_t(n) = T(n/3) + 4c$$

$$T_B(n) = 2c \log_2 n$$

$$T_t(n) = 4c \log_3 n$$

$$2c \log_2 n < 4c \log_3 n$$

$\therefore$ Binary search is less costly + more efficient than ternary search.

Q. $n=64$, $T(n) = 305 \Rightarrow n$ for $T(n) = 3608$?

A. $T(n) = c \cdot n \cdot \log n$

$$536 = c \cdot 64 \cdot \log 64 = c \cdot 64 \times 6$$

$$c = 5/64$$

$$\therefore \frac{3608}{64} = \frac{5}{64} \times n \times \log_2 n$$

$$\text{i.e. } n \times \log_2 n = 72 \times 64$$

$$n \times \log_2 n = 9 \times 512$$

$$\therefore n = 512$$

$$\log_2 n = 9$$

$\therefore$ max. input size = 512 elements

Q. 20, 47, 15, 8, 9, 4, 40, 30, 12, 17. straight two-way merge sort
(ignore the word straight)

A. P1) 20, 47, 8, 15, 4, 9, 30, 40, 12, 17

P2) 8, 15, 20, 47, 4, 9, 30, 40, 12, 17

$\therefore$ 8, 15, 20, 47, 4, 9, 30, 40, 12, 17.

10/01/24



• Quicksort Algorithm

→ Given an array of n elements,

- if 1 element, return

- if > 1 element, choose a pivot element, then split array into:
elements $\leq$ pivot
elements $>$ pivot

- Quicksort the new arrays

• Return results.

pointer 1 (next elem)

pointer 2 (last element)

Ex. 40 20 10 80 60 50 7 30 100

↓
select first num as pivot,
place it in the position
where

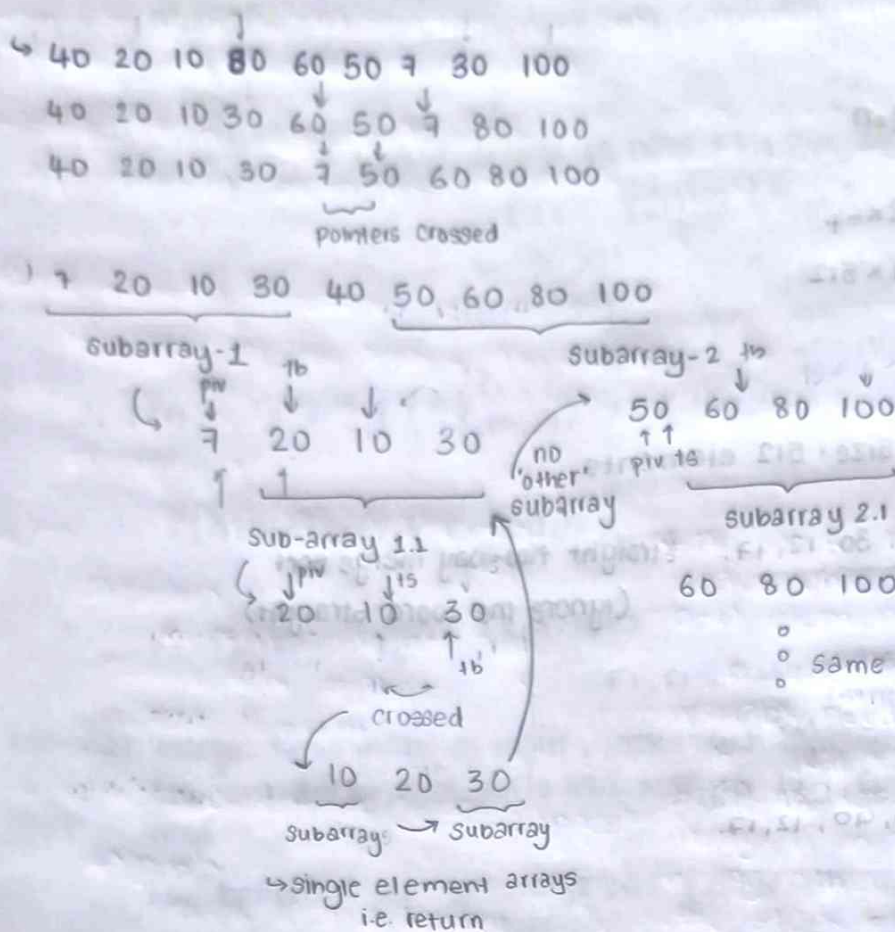
nums < 40 40 nums > 40

pointer 1 moves rightwards as long as it points to a value < 40 , and stops when it reaches a number > 40

pointer 2 moves leftward similarly till we reach < 40

then, values pointed by 1 and 2 are swapped, and we continue.

we stop moving the pointers once they cross each other and swap values of pivot and pointer 2



→ what will the time complexity be if we somehow manage to choose the middle element as the pivot?

$$T(n) = \underbrace{T(n/2)}_{\text{subarray 1}} + \underbrace{T(n/2)}_{\text{subarray 2}} + \underbrace{cn}_{\text{swaps}}$$

$T(n) = n \log n$ in this case

$$T(n) = 2T(n/2) + cn$$

$$\Rightarrow T(n) = n \log n$$

if element was first / last:

$$T(n) = T(n-1) + cn \quad (+T(0), \text{ which is } = 0)$$

$$T(n) = T(n-2) + c(n-1) + cn$$

$$T(n) = T(1) + c(2+3+4+\dots+n)$$

$$T(n) = T(0) + c(1+2+3+\dots+n)$$

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2} \approx \frac{n^2}{2} \approx n^2 \quad (c' = c/2)$$

$n^2/2$ is equivalent to n^2

since $-n/2 < n^2/2$, we can ignore it.

→ the time complexity of quicksort tends to n^2 , and the more sorted the array, the more time complexity it has.

→ To improve pivot selection, we choose pivot as the median of $\text{data}[0]$, $\text{data}[\frac{n}{2}]$ and $\text{data}[n-1]$.

Q. Possibility that pivot gets placed in worst possible location in first round of partitioning when sorting 25 elements using quicksort.

A. worst possible location = first element / last element

$$= \frac{2}{25} = 0.08$$

Q. If input of sorting is already sorted:

1. Quicksort runs in $O(n^2)$ time

2. Bubblesort runs in $O(n)$ time $\Rightarrow$ only one iteration, we break if no swaps made.

or $O(n^2)$ time if not optimised

$\hookrightarrow$ optimised bubble sort

3. Mergesort runs in $O(n \log n)$ time. $\Rightarrow$ mergesort is always $n \log n$

4. Insertion sort runs in $O(n)$ time

Q. How many swaps in selection sort in worst case?

A. n swaps.

Q. Given an unsorted array of integers, $\rightarrow$ expected: $O(n \log n)$
give the maximum perimeter.

A. 1. sort it using merge sort

2. Go through from the last element, finding

first element s.t. the sum of the elements after it $>$ element itself

i.e. $[33, 6, 20, 1, 8, 12, 5, 55, 4, 9]$

$\downarrow$ merge sort

$[1, 4, 5, 6, 8, 9, 12, 20, 33, 55]$

$\begin{array}{l} 55 > 20 + 33 \\ \downarrow \\ 33 > 12 + 20 \\ \downarrow \\ 20 < 12 + 9 \end{array}$

$\therefore [20, 12, 9]$

$$\therefore \text{perimeter} = 20 + 12 + 9 = 41$$

Thus, we ensure maximum + that sides actually form a triangle.

Since 1 is $O(n \log n)$, 2 is $O(n)$, overall complexity is $O(n \log n)$.

17/01/25

Stacks

→ called an abstract data structure: it is only concerned with executing the operation, not how it is executing.

→ push: adding elements to stack → you cannot push to a full stack
pop: removing elements from stack → you cannot pop an empty stack

→ implementing operations via arrays:

1) constructing the array + flag variable

↳ i.e. set $myTop = -1$

2) empty():

check if flag variable = -1

3) push():

check if array not full

increment $myTop$

add element

↳ else: output "out of space"

4) pop():

check if array empty

print current element

decrement $myTop$.

↳ else: output "nothing to pop"

★ The stack can have space while array has $(n-1)$ elements:

because the flag variable need not be equal to $n-1$ i.e. be at the top of the array. If $myTop$ is at any other position, the stack has space even if the array is technically filled.

→ not an issue if we use linked lists.

⇒ what happens if you run out of space in the array i.e. initial array taken is too small?

option i) everytime you run out of space, add a fixed 'c' amount of space

option ii) everytime you run out of space, double the current amount of space.

→ it takes 'nc' operations in the n^{th} step to execute this

$$\therefore \frac{c(n)(n+1)}{2}$$

operations in total i.e. $O(n^2)$ time complexity.

includes operations req. for:

- 1) creating new array
- 2) copying old elements
- 3) filling new elements

→ it takes $c \cdot 2^{(n-1)}$ operations in the n^{th} step

$$\therefore O(n \log n) \text{ time complexity}$$

∴ option ii is better.

→ Applications:

1) Balancing parenthesis:

Ex: $([])\{([\{ \}] \{ \})\}$. suggest an algorithm that determines whether the brackets are balanced or not.

→ Iterate through, everytime you see a new character:

- i) if it is an open bracket, push it to the top of the stack
- ii) if it is a closed bracket, check the top of the stack for the corresponding open bracket.

if you find the correct bracket in the stack, pop it. else, return an error

- iii) after iterating through the entire string, if there are any elements remaining in the stack, return an error.

2) converting infix → postfix

$a+b$

$ab+$

It is easier to write algo that tells a computer to solve postfix notation.

- Ex)
- a) $(a+b-c)*d-(e+f) = ab+c-d*ef+-$
 - b) $4^2*3-3+8/4/(1+1) = 42^3*3-84\div+1+\div+$
 - c) $a+b/c*(d+e)-f = abc\div de+*+f-$

→ Iterate through, and:

- i) if you see operand, append to end of output list
- ii) if you see '(', push it onto the stack
- iii) if you see ')', pop the stack till you have popped the left parenthesis '('. Append the operators popped to the end of the output list.
- iv) if you see an operator, push it on the stack. Before that, append any higher precedence operators to the end of the output list, or equal and pop them from the stack.

Precedence:

$\wedge > */\div > +/ -$

At the end of the iteration, any elements still on the stack can be popped and appended to the output list.

Ex) b) $4^2*3-3+8/4/(1+1) = 42^3*3-84\div+1+\div+$

d) $A^b^c = A^b^c$ → acc to above method, $Ab^c^$
→ actual answer: $Abc^{\wedge\wedge}$

*exception:

∴ in the case of exponentials, only pop things of higher precedence (NOT higher and equal) before placing on stack.

→ to solve postfix notation using stack, push operands on stack, and when you encounter an operator, pop the previous two elements on the stack, apply the operator, and push the result back onto the stack.

once we iterate through the entire string, popping the final element gives the final result

21/01/24

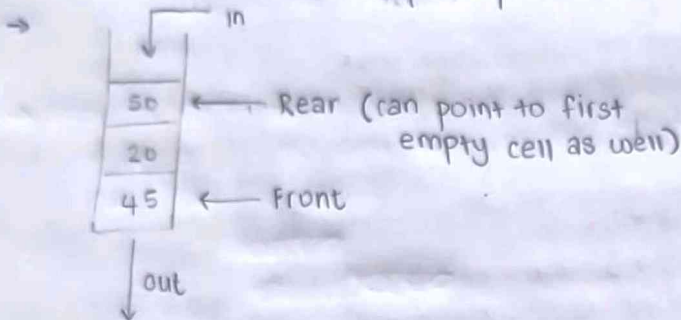
Queues

→ Enqueue: entering the queue

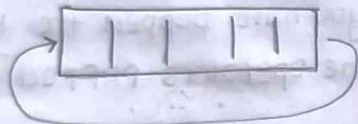
Dequeue: leaving the queue

→ First-in, first-out principle is followed

Implement stacks and queues



As with stacks, queues can have space even if the array it uses is full.



pointers should be able to shift from the last to the first element

→ For a linear queue,

i) Enqueue:

```
enqueue(queue) {
    if (R == (n-1)) // full
    } else {
        if (F == -1) {
            F++;
        }
        R++;
        queue[R] = a;
    }
}
```

ii) Dequeue:

```
dequeue(queue) {
    if (F == R+1 || F == -1) {
        // empty
    } else {
        printf("%d", queue[F]);
        F++;
    }
}
```

→ For a circular queue,

i) Enqueue:

```
enqueue(queue){
    if ((R+1)%n == F && R > F){
        // queue full
    } else {
        if (F == -1){
            F++;
        }
        R++;
        scanf("%d", &queue[R]);
    }
}
```

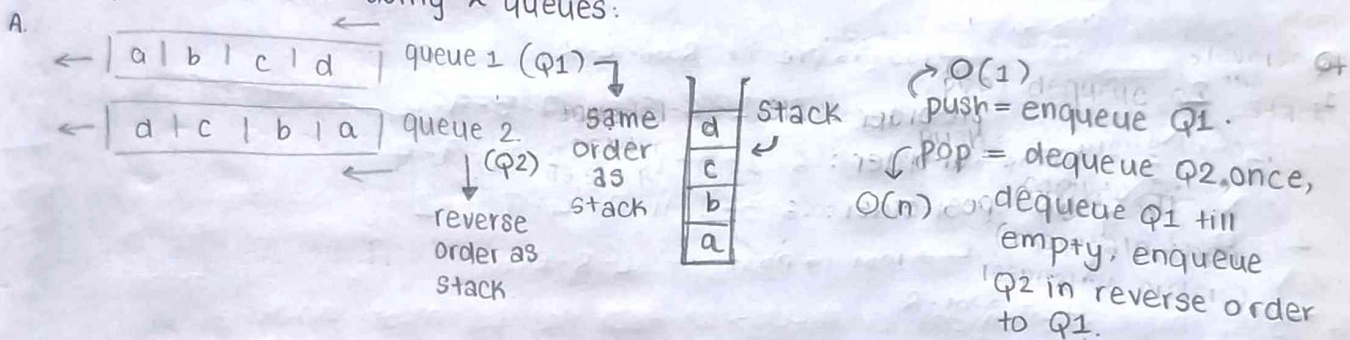


ii) Dequeue:

```
dequeue(queue){
    if (((R+1)%n == F && R < F) ||
        // queue empty F == -1){
    } else {
        printf("%d", queue[F]);
        F++;
    }
}
```

→ if we use linked lists to implement queues, we need not use circular linked lists, but if we use arrays, we need to use circular arrays.

Q. Implement a stack using 2 queues:



to print stack, print Q1.

we can make popping the cheaper operation instead as well

Q. Implement a queue using two stacks:

A.

24/01/25

→ A graph is a set of nodes which are connected such that $\exists$ a root node, children of the root node and the connections here are called 'edges'.

If there is one unique path from the root node to any node in the graph, it is called a tree.

→ A Binary Tree is one where each node has at most 2 ~~children~~ children.

→ i) depth of a node = ~~max~~ no. of edges in the unique path from the root node to it

ii) height of a node = length of the longest path from a node to a leaf node that is its child.

iii) If n_2 is along the path of one of a children of n_1 , n_1 is an ancestor of n_2 .

→ implementation:

→ when we do not know no. of children of nodes, we can define a

~~structure~~

~~structure~~

struct TreeNode {

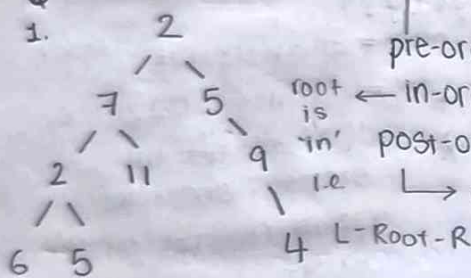
struct* TreeNode* FirstChild;

struct* TreeNode* Next_Sibling;

}

Q

1.



→ root is pre i.e. Root-L-R

pre-order: 2-7-2-6-5-11-7-5-9-4

In-order: 6-2-5-11-7-2-5-9-4

post-order: 6-5-2-11-7-4-9-5-2

→ root is post i.e. L-R-Root

L-Root-R

→ level-order: elements in the same are taken left to right.

2. 1, 5, 1, 8, 3, 6, 0, 9, 4, 2 added in seq. to Binary Search Tree. are on what is ~~its~~ its inorder traversal?

→ A binary tree where no.s

<root are on the left and > root

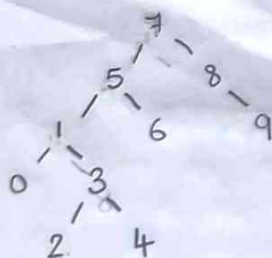
are on the right

→ in-order traversal of a BST always returns sorted order of elements

in-order: ~~0-1-2-3-4-5-6-7-8-9~~

0-1-2-3-4-5-6-7-8-9

(i.e. in ascending order)



3. pre-order traversal: ABECD $\rightarrow$ root-left-right
 in order traversal: BEADC.

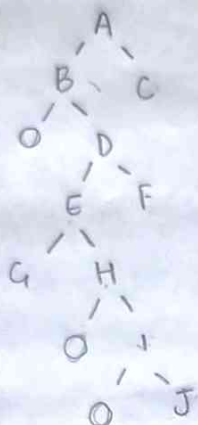
level order sequence?



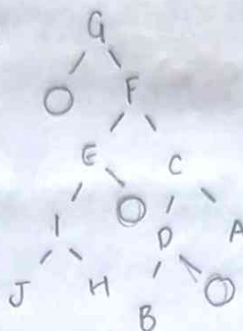
$\therefore$ level order sequence = A-B-C-E-D

★ Given in-order +
 post-order &
 pre-order + post-order,
 can you find the
 tree?

4. Which ordering of



and



gives same answer?

pre: A-B-D-E-G-H-I-J-F-C
 in: B-G-E-H-I-J-D-F-A-C
 post: G-J-I-H-E-F-D-B-C-A

pre: G-F-E-I-J-H-C-D-B-A
 in: G-J-I-H-E-F-B-D-C-A
 post: J-H-I-E-B-D-A-C-F-G

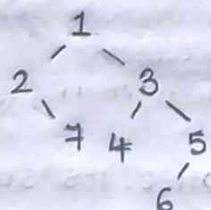
none of them match :-

5. In order ~~traversal~~ traversal of a tree: FBQADCE. Pre-order ~~traversal~~ traversal of same tree would be?

- a.
- b.
- c.
- d.

★ note down options and try

6. Pre order: 1, 2, 7, 3, 4, 5, 6, 5 in order: 2, 7, 1, 4, 3, 6, 5. what is the tree?



$\rightarrow$ get root from pre-order, and use Φ in-order to find all the elements on the root and all the elements on the right

i.e. if in order = a b c d e

left of root
 in tree

root

right of root
 in tree

→ A proper binary tree is a tree in which every ~~other~~ node has exactly 2 or 0 children nodes.

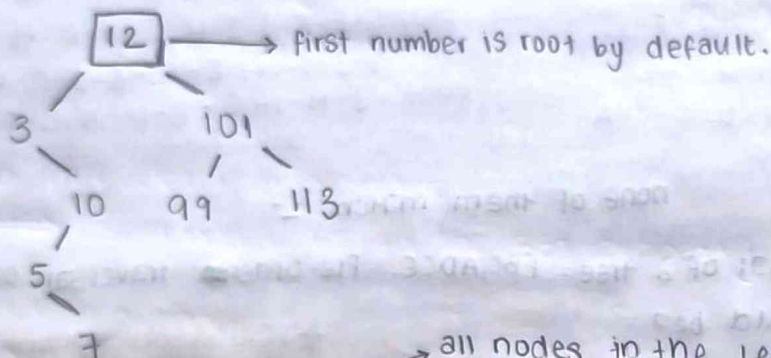
Q. pre-order: 1, 2, 3, 4, 5, 6, 7
 post-order: 2, 4, 6, 7, 5, 3, 1 } for a full/ proper BST.
 construct the tree



_____ X _____

Q. Create a BST using the numbers: 12, 3, 10, 5, 7, 101, 99, 113

A.



all nodes in the left subtree of a node are smaller than it, and all nodes in the right subtree are greater

→ the smallest element is always at ~~the~~ the leftmost node.
 similarly, the largest element is always at the rightmost node.

→ the ~~successor~~ successor / predecessor is the number immediately ~~before/after~~ after / before the given number in the sorted list of numbers.

to find successor / predecessor:

s: i) ~~if a node~~ if a node has a right subtree, its successor is the leftmost node of that subtree. ii) If the right subtree of the node doesn't exist, we must traverse backwards through the tree.

- i) If we are on the left subtree, the first node that has a left subchild is the root. At worst, this node is the root.
- ii) If we are on the right subtree, the first node with a left subchild is the successor. If there is no such node in the entire right subtree, the node we have has the greatest value, and successor does not exist.

(if at root in case (i), root = successor
(ii), successor doesn't exist.

p: (i) If ~~the~~ the node has a left subtree, check for the rightmost node in that subtree to get predecessor.

(ii) If the left subtree of the node doesn't exist, we traverse backwards in the ~~main~~ main tree, and try to find the first node which has a right subchild. This node is the predecessor.

If we are on the left subtree and reach the root in ~~the~~ this manner, we say the predecessor doesn't exist.

→ Basic operations like finding minimum/maximum node, successor, predecessor ~~are~~ take time proportional to the height of the tree

i.e. $O(h)$

→ This method is preferable to doing in-order traversal, then going through the result (sorted list) for single / small no. of queries ~~as~~ as the first process is $O(n)$ best case while second process is $O(n)$ worst case.

for a small no. / 1 query.

→ Inorder traversal of a BST:

Inorder(x)

if $x \neq \text{NIL}$; → case to ensure tree exists

then Inorder(Left[x]) → inorder traversal of left subtree

print key[x] → print element.

Inorder(Right[x]) → inorder traversal of right subtree

→ Tree Search: → root

⇒ Tree-search (x, k) num to search for.

if $x = \text{NIL}$ or $k = \text{Key}[x]$ } to check if $x = k$ / x doesn't exist,
then return x

if $k < \text{Key}[x]$

then return Tree-search (Left[x], k)

→ if $k < x$, search left subtree of x for k

else return Tree-search (Right[x], k)

→ if $x < k$, search right subtree of x for k

⇒ This process runs in $O(h)$.

⇒ there is also an iterative approach to tree search, which is more efficient

~~• Recursive approach~~

→ To find minimum / maximum, keep going to the left / right subchild of each node. If the current node has no left / subchild, then it is the the minima / maxima at any point.

→ Insertion and deletion of nodes

⇒ Insert nodes acc. to the BST property.

→ deletion of nodes: ^{this is the way that has} minimal computation.

i) if the node has only one child, link parent of that node to the child of the node.

→ takes up the same spot the parent did.

ii) if the node has no children, just delete it.
i.e. leaf

iii) if the node has 2 children, replace current node with successor, and ~~delete~~ delete the successor.

→ it is either case (i) or (ii), so we know already how to delete it.

04/02/25 -

Fenwick trees - Binary indexed trees implicit data structure

A

4	3	8	11	20	6
0	1	2	3	4	5

→ Given two indices $0 \leq l < j \leq n-1$, compute the sum of elements from ~~array~~ $A[l]$ to $A[j]$

(ex: if $l=2, j=4$, output = 39)

m1) for (sum=0, x=l; x≤j; x++) {
 sum += x;
}

runtime = $O(n)$ in worst case.

→ the diff b/w m1 and m2 is that for multiple queries, $O(Kn)$, $O(n)$.

m2) S = 2D array of sum.

$S[l][j]$:

$A[l] + A[l+1] + \dots + A[j]$

	0	1	2	3	4	5
0	4	7	15	26	46	52
1	x	3	11	22	42	48
2	x	x	8	19	39	45
3	x	x	x	11	31	37
4	x	x	x	x	20	26
5	x	x	x	x	x	6

(say: k)

Some observations:

i) $\forall i=j$, original array is present

ii) using this is $O(1)$ time,

but $O(n^2)$ space complexity

iii) $S[0][j] = S[0][i-1] + S[i][j]$

$\therefore S[i][j] = S[0][j] - S[0][i-1]$

$$S[i][j] = S[0][j] - S[0][i-1]$$

→ $\therefore$ now, we only need $S[0][x]$

let us create $S' = S[0]$

$S' =$

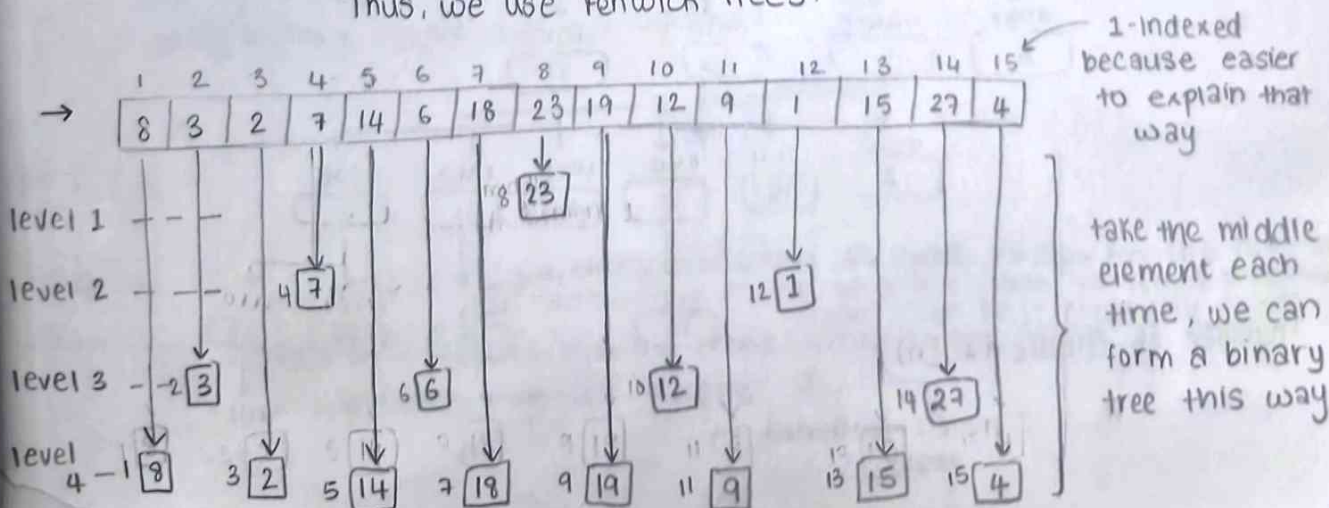
4	7	15	26	46	52
---	---	----	----	----	----

→ $O(n)$ space complexity.

creating this array S' would take $O(n)$ time.

if one of the entries of A changes, we will have to repopulate the entirety of S' (worst case) i.e. a new $O(n)$ operation every single time.

Thus, we use Fenwick trees:



level 1: 1000 [3]
 level 2: 0100 1100 [2]
 level 3: 0010 0110 1010 1110 [1]
 level 4: 0001 0011 0101 0111 1001 1011 1101 1111 [0]

we
 % Fill a new array with the same structure, where each element is a sum of the $2^{[n]}$ elements before it (including that element itself) in the original array.

i.e. ~~array~~

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
8	11	2	20	14	20	18	81	19	31	9	41	15	42	4

query tree

Fenwick tree

% let us say we want to calculate 7th index:

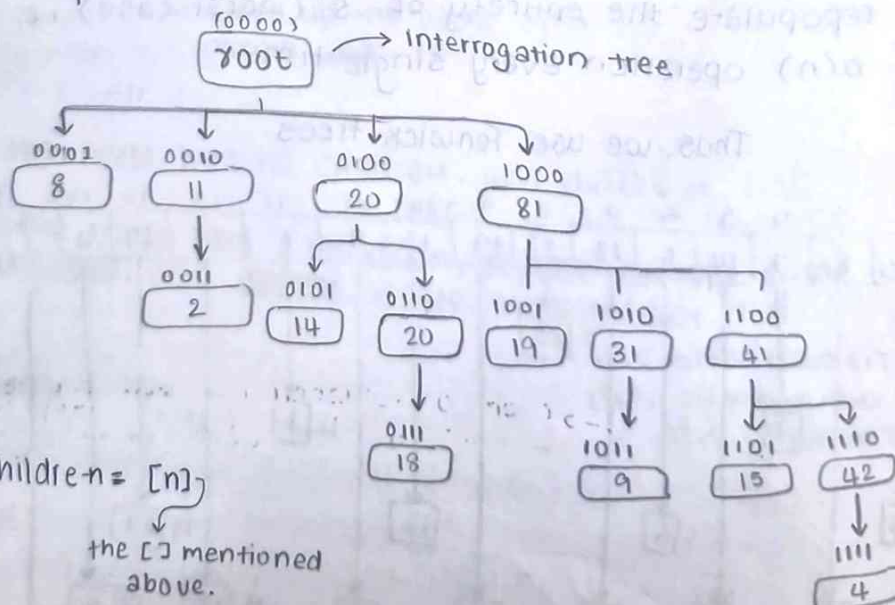
step 1: at 18, go back 1 indices index
 step 2: at 20, go back 2 indices i.e. go back $2^{[n]}$ indices
 step 3: at 20, go back 4 indices

Now, we have gone to the start of the array.

% sum = 18 + 20 + 20 = 58.

similarly, we can calculate for any other index.

→ Interrogation + update tree:

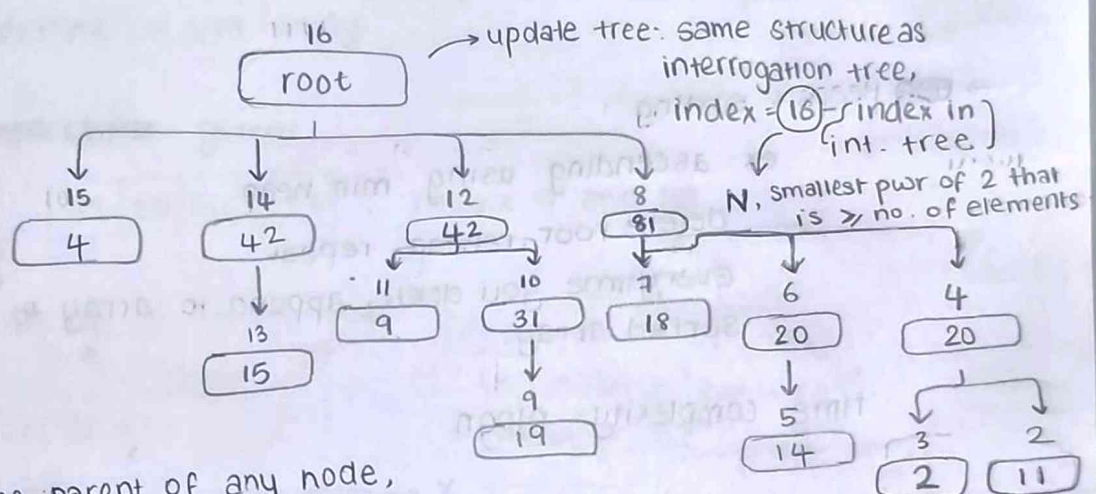


to find the parent of any node, take the binary, convert the rightmost 1 and make it 0, and you have the binary of the parent's index

∴ the interrogation tree is NOT an actual structure we create. It is a method of querying the ~~tree~~ Fenwick tree (also not a tree data structure, it is an array data structure)

∴ to find the sum of any element, start with that node, and add the values of all the nodes you find while going from that node to the root via the shortest path

i.e. node $\rightarrow$ its parent node, as long as node $\neq$ root.



to find the parent of any node, ~~find corresponding node in int. tree~~ find its parent in that tree, and find corresponding node in the update tree.

Here, n is conveniently 15, to preserve structure 1 when n has other values, we use dummy nodes.

$\rightarrow$ How do you create this?

for $O(n)$ time, ^{for every entry} update a node and its immediate parent. This way all the nodes will get updated.

———— X ————

07/02/25

• Heapsort

$\rightarrow$ min/max heap: A complete binary tree where all levels except for the last are completely filled, and the last is filled left to right (can be incompletely filled) greater than its parent

→ sift up: moving a larger element up to give a node the heap property

→ insertion into a heap: insert node in tree, sift up to appropriate position.
Takes $n \log n$ worst time complexity.

→ deletion from a heap: delete node, replace with rightmost node of last level, convert into heap again using sift up.
↳ if you delete the root, the the rightmost node in the last level, fix it in root position, and sift up to recreate the heap

→ ~~Heap~~ Heaps + sorting :

ex: ascending using min heap.

delete root, reheap, repeat.

everytime you delete, append to array to get sorted array.

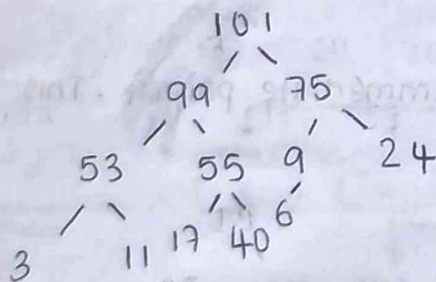
time complexity = $n \log n$

11/02/25

→ Ex:

arrange in desc. order: 17, 24, 55, 3, 75, 9, 101, 53, 11, 99, 40, 6

Step 1: ~~add~~ insert into heap:



Step 2: continuous deletion + reheapify-ing of heap created.

101, 99, 75, 55, 53, 40, 24, 17, 11, 9, 6, 3.

Q. Arrange the given heap in an array such that we can easily reference the child of a given node / parent of a given node with index only.

A. → left child and right child of i will be at $2i+1$ and $2i+2$ respectively.

→ parent of a node located at i is at $(i-1)/2$

If we are trying to implement a Binary tree, rather than complete binary tree, we leave empty spaces in our array to replace the non-existent elements

∴ now, there is no need of a tree at all, and we can continue our operations on this array.

Ex) ~~insertion~~ given:

75	55	24	3	17
----	----	----	---	----

, insert 9 and 101.

75 55 24 3 17 9 101

↔ swap

75 55 101 3 17 9 24

↔ swap

101	55	75	3	17	9	24
-----	----	----	---	----	---	----

i.e. insert at last index,
check values with parent until

a) a swap isn't done

b) $i=0$ i.e. element is at root.

→ $O(n)$ since it is just iteration through the array.

⇒ deletion from this array:

i) delete 0th element, replace with last element

55 75 3 17 9 24

↓

24 55 75 3 17 9

↔ swap

75	55	24	3	17	9
----	----	----	---	----	---

i.e. we need to keep a pointer to the last element of the array and update it as and when required

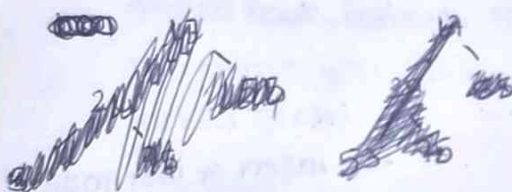
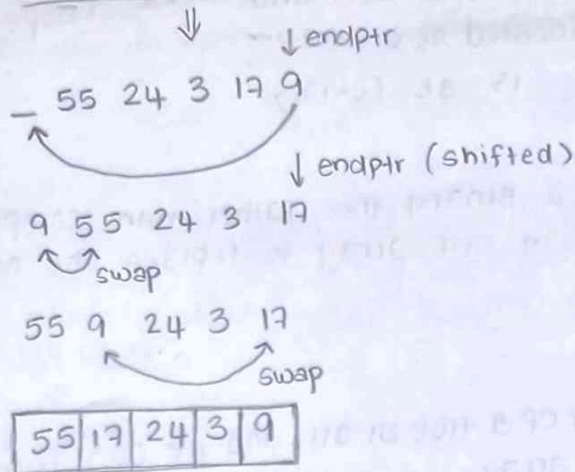
ii) check values with children and swap until

a) no swap is made

b) element is the last element

Ex: one more deletion from array:

75 | 55 | 24 | 3 | 17 | 9



18/02/25

• Heap Algorithms:

a) Parent ($A[i]$) a heap indexed $1 \rightarrow n$

if ($i == 1$)
return NULL;

return [$i/2$]

b + c) left and right ($A[i]$)

→ finding left and right child via index $2i$ and $2i+1$.

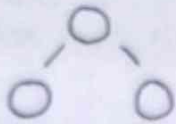
* Check heaps-pseudocode document.

• AVL trees

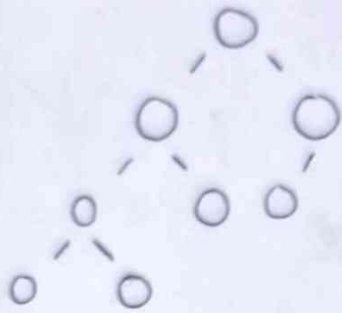
→ A binary search tree is AVL balanced if:

- i) difference in heights of left and right subtrees is at most 1
- ii) left and right subtrees are also AVL trees.

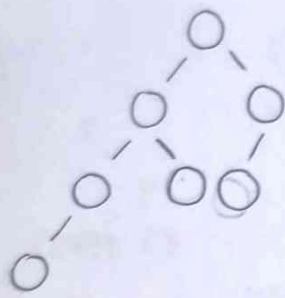
Ex. AVL tree of height 1:



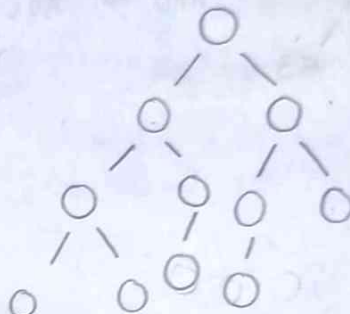
AVL tree of height 3



Ex: AVL tree of height 3 and 4 w/ minimum no. of nodes.



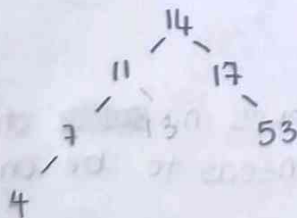
height = 3,
nodes = 7



height = 4
nodes = 12

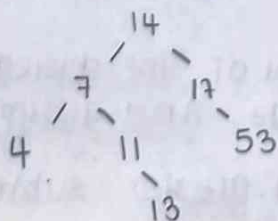
→ Let $n(h)$ be minimum no. of nodes in a tree of height h ,
then $h \leq 2 \log_2 n(h)$ check derivation in slide

Ex) inserting 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree.

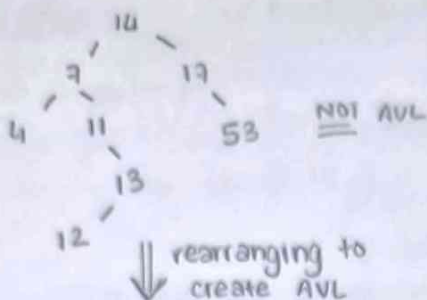


→ after inserting 4, the tree is no longer an AVL tree

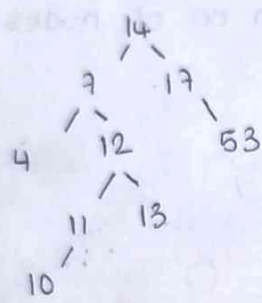
⇓ rearranging elements to create a new AVL tree.



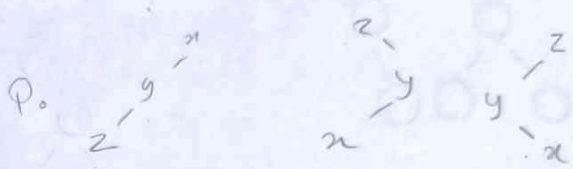
now, inserting 12.



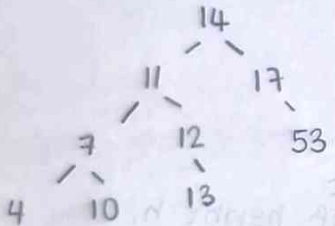
Ex) Convert



into an AVL tree



work out these



✧ how to rearrange trees s.t. they are AVL?

24/02/25

→ On deleting a node from an AVL tree, if it makes no ~~change~~ change in the height of the subtree, then nothing needs to be changed.

However, if the height of the subtree changes,

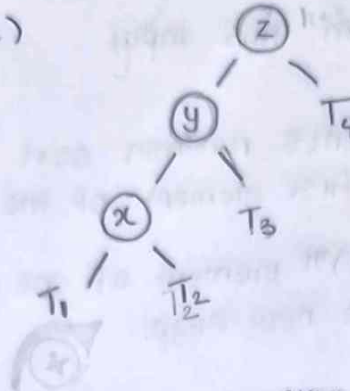
we:

- 1) traverse backwards from the parent of the deleted node, and find the first unbalanced node, and assign it 'z'
- 2) we find the child of z that has a greater subtree, and label it 'y'. → y will never be an ancestor of the deleted node, as height decreased when it got deleted.

3) we find the child of y with greater ~~value~~ height, and label it ' x '.
If both subtrees have the same height, choose the straight line.

i.e.

(case 1)



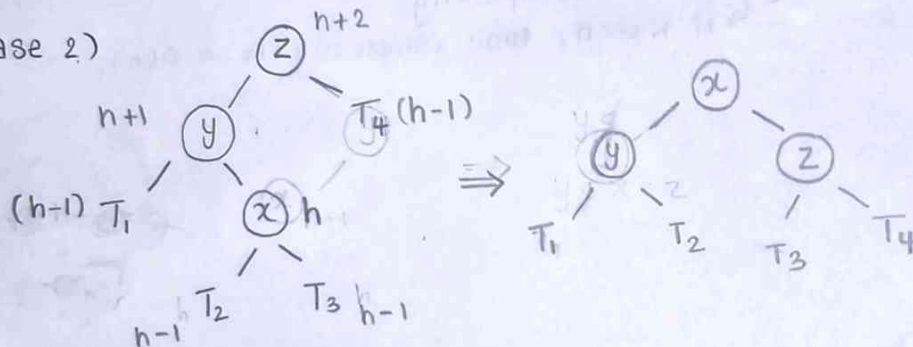
Now, we can ~~rearrange~~ arrange these nodes :



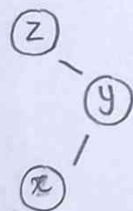
rearrange nodes s.t.
 $x < y < z$ (this is for any
[By value] case),

and fill in the other nodes.
(by checking values)

(case 2)



(case 3)



} solved + balanced the same way
as in case 2.

→ if balancing this subtree ~~causing~~ causes another imbalance, we repeat this process all over, and fix that imbalance.

check by traversing back
till root

→ why should we choose the straight line for x when both subtrees have the same height: because choosing that node may cause an imbalance in y after rearranging, which we do not want.

Q Given an array is K -sorted,
 i.e. an element is at most K positions away from its actual position

Ex: $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 2 & 3 & 1 & 5 & 4 & 6 & 8 & 7 \end{matrix}$
 K -sorted $\underbrace{\hspace{1.5cm}}$ 2 away

create the sorted array from this input.

A.

We take the first $(K+1)$ elements, run min sort once, obtain min element, and that is the first element of the sorted array.

After this, we insert the $(K+2)^{\text{th}}$ element of the input array, and repeat the process with the new heap.

We continue till there are no elements to insert, and then we get the sorted array by doing full heap sort.

i.e. there are $(n-K)$ insertions $\Rightarrow (n-K)\log K$
 n deletions $\Rightarrow n\log K$

$\therefore O(n\log K)$ time complexity
 if $K \ll n$, time complexity is $\approx O(n)$.

03
 04/12/25

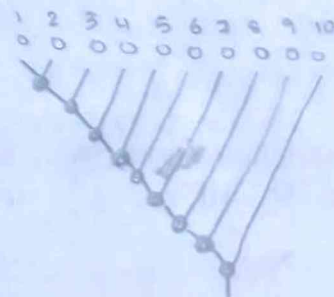
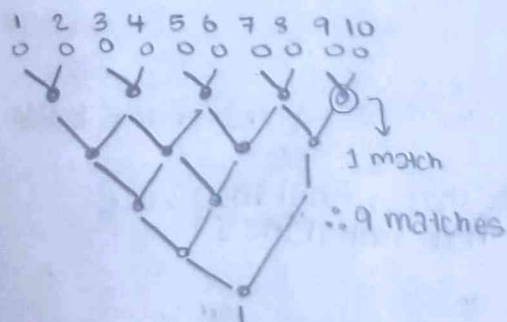
Q. There are n tennis players. How many matches are required to find best player?

(Hint: if A is better than B , and B is better than C , then A is better than C)

A.

Ex: Let $n=10$

$\rightarrow 9$ matches?



→ 8 matches?

1 2 3 4 5 6 7 8 9 10
0 0 0 0 0 0 0 0 0 0

→ sorting
→ stacks + queues
→ Bst
→ heaps
→ AVL trees

you cannot. Take player 1. He needs to have played

~~9~~ 9 matches (one against every other player) to determine for sure that he is the best, as he needs to have won against every other player.

If only 8 matches are played, there will be 8 players who lose and are ~~not~~ not the best. But, the remaining 2 players are unaccounted for, ∴ 1 more match is required

i.e. 9 matches is the minimum.

↓
equivalent to

how many comparisons are required to find the maximum of n elements in an array?

$$= \boxed{n-1 \text{ comparisons.}}$$

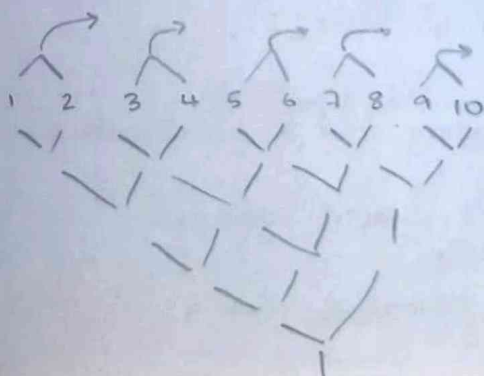
Q. How many comparisons are required to find maximum and minimum of n elements in an array?

$$A = (n-1) + (\lceil n/2 \rceil - 1) = \boxed{\lceil 3n/2 \rceil - 2 \text{ comparisons}}$$

↓
to find
maximum among
the elements

↓
to find the
minimum of
the $n/2$ elements
that "lost" in the
first round

i.e. let $n=10$

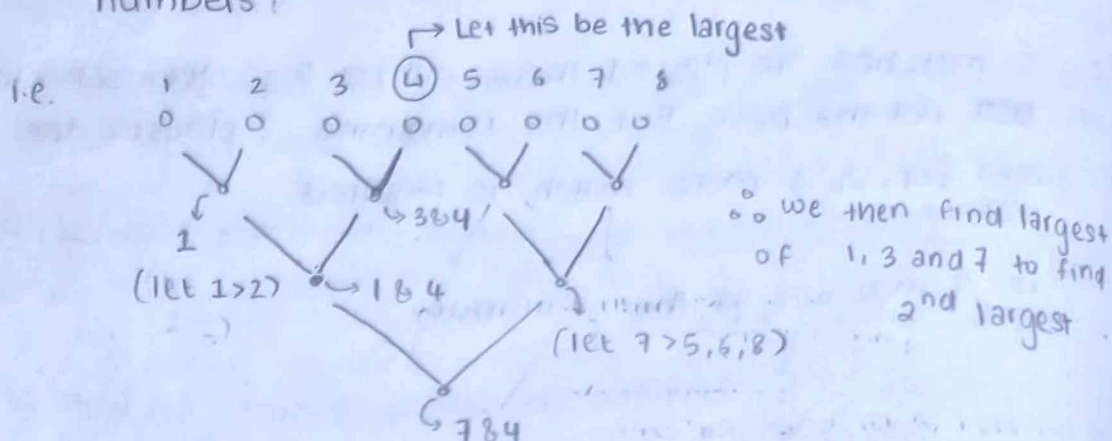


⇒ 1' 2' 3' 4' 5' } finding the max and min separately takes the same number of steps

Q. How many comparisons are required to find largest and second-largest number?

A. → The only ~~number~~ number the second largest number is less than is the largest

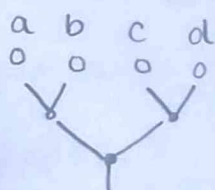
i.e. we only need to consider the numbers in the comparison with the largest, and take the largest of those numbers.



$$\therefore (n-1) + ([\log_2 n] - 1) \text{ comparisons}$$

Q. If you have 4 players, how many matches would it take to find relative rankings of the players?

A) → matches to find best and worst : 4
matches to find best + second-best : 4

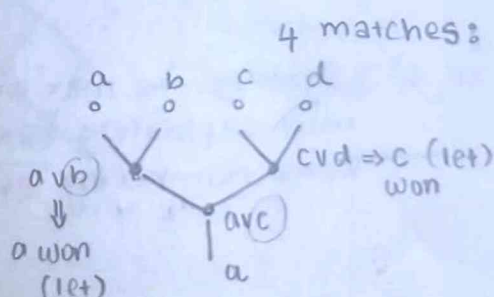


5 matches:

4 to do best + worst → i.e. rank 1 and 4

1 to find best of remaining two → i.e. rank 2 and 3

∴ ranking found



ONLY works if $b > c$. if $c > b$, it takes more than 4 matches

3 matches to find $a \Rightarrow$ rank 1

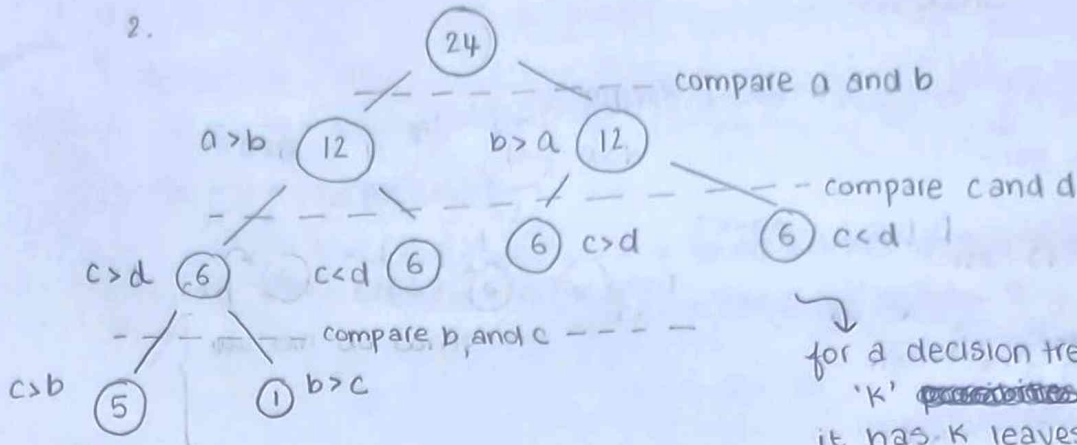
1 match to compare b and $c \Rightarrow$ rank 2

now, whoever played against rank 2 is rank 3, and the last element is rank 4.

↳ decision tree for this algorithm :

1. number of possible rankings $\Rightarrow n! = 4! = 24$

i.e. there are 24 possible answers



↳ i.e. in this case, it takes more than 4 matches to find ranking

i.e. it is impossible to find ranking of 4 numbers in only 4 matches

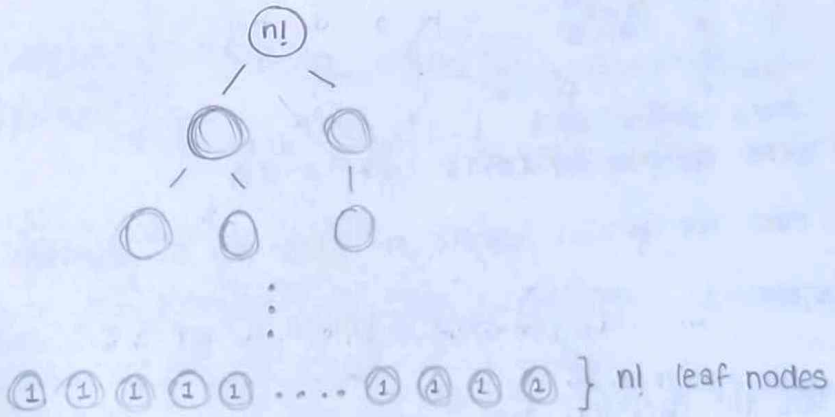
↳ Th, It is impossible to find the relative rankings of 4 players by playing 4 matches

for a decision tree that starts w/ 'K' possibilities, it has K leaves, each with value '1'.

If there are K nodes at depth d, there are at least $\lceil K/2 \rceil$ nodes at depth 'd-1'.

→ sorting in numbers:

decision tree for sorting ~~n~~ n numbers using S, the best possible algorithm :



depth of this tree $\geq \log_2(n!)$

$$\geq \log_2 (n \times (n-1) \times \dots \times (\frac{n}{2}+1) \times (\frac{n}{2}) \times (\frac{n}{2}-1) \times \dots \times 3 \times 2 \times 1)$$

$$\geq \log_2 \left(\underbrace{\left(\frac{n}{2}\right)^{\frac{n}{2}} \times \dots \times (\frac{n}{2}+1) \times (\frac{n}{2}) \times \dots \times 2^2 \times 2}_{\frac{n}{2} \text{ terms}} \right)$$

$$\geq \log_2 \left(\underbrace{\frac{n}{2} \times \frac{n}{2} \times \dots \times \frac{n}{2}}_{\frac{n}{2} \text{ times}} \times \underbrace{2 \times 2 \times \dots \times 2}_{\frac{n}{2} \text{ times}} \right)$$

↳ replacing every term w/ smthg $\leq$ it

$$\geq \log_2 \left((n/2)^{1/2} \times (2)^{1/2} \right)$$

$$\geq \log_2 (n^{1/2})$$

$$\geq \frac{n}{2} \log_2(n)$$

$$\geq \frac{1}{2} (n \log_2 n) \text{ or } \geq c' n \log_2 n, c' = 1/2$$

∴ order(s) = $\Omega(n \log n)$

$$\frac{1}{2} n \log n \leq |S| \leq c n \log n \quad \rightarrow c \geq 1$$

ie. any comparison based sorting algorithm must have at least $\frac{1}{2} n \log n$ steps $\Rightarrow$ proven by making the decision tree

$\hookrightarrow$ every algo has a decision tree, no exception

Q. How many comparisons are required to sort 6 numbers (distinct)?

A.

$\rightarrow$ It is possible to sort 5 distinct numbers in 7 comparisons

1 2 3 4 5

Step 8. compare w/ 3

Step 9+10. compare w/ 1 and 2 (or) 5 and 4

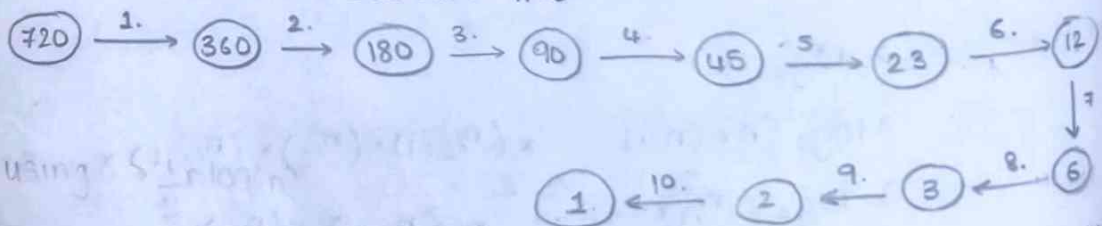
or

now, we know where to place the 6th element

∴ 10 steps

$\rightarrow$ using $\geq \frac{1}{2} n \log n$, we get ≥ 8.75 , then:

$\rightarrow$ we can also use the decision tree:



using $\frac{1}{2} n \log n$

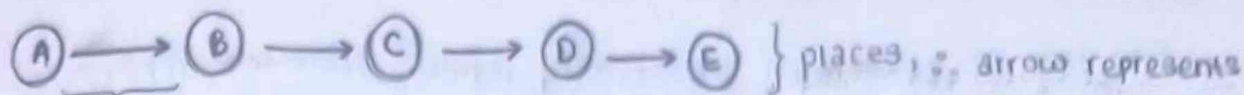
we get minimum

After 9 steps, there is still a node that is not = 1

∴ 10 is the minimum no of required steps

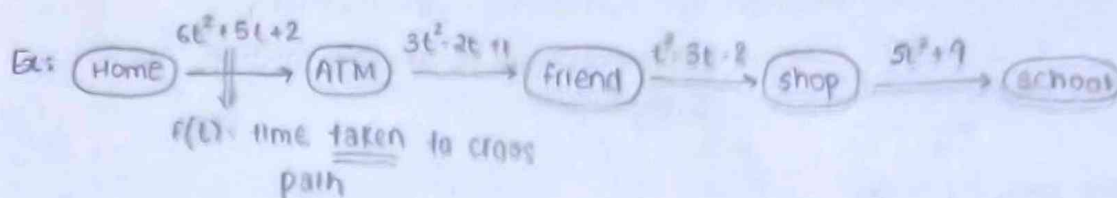
07/03/25

→ consider

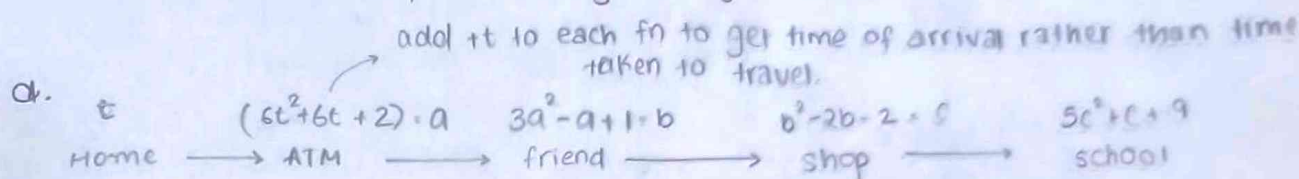


time variant i.e.
time taken for this
is not constant, and
can change

time taken to go from
one place to another



- Q. What is the fn of time to reach @ from home to school?
- Q. Does the fn have a closed form expression?
- Q. Is the function a polynomial in t ?
- Q. Can it be computed using the given information?



1. $5c^2+c+9 \Rightarrow$ plug in values as required

2. Yes

3. Yes

4. Yes

The degree of the polynomial is: 16

if there are " k " paths,
degree would be 2^k .

i.e. composition of quadratics
 k times gives 2^k

composition of linears k
times gives 1

i.e. composing multiple functions of linear
time will give us a final function that
has linear time as well.

→ We can now find an algorithm s.t. we find the median of an unsorted array in $O(n)$ time, using the above statement.

→ Consider:

3	9	1	7	4	6	2	8	5
---	---	---	---	---	---	---	---	---

i. to find the position of an arbitrary number from the array, iterate through the array, and keep count of how many elements are less than ~~and~~ / greater than it.

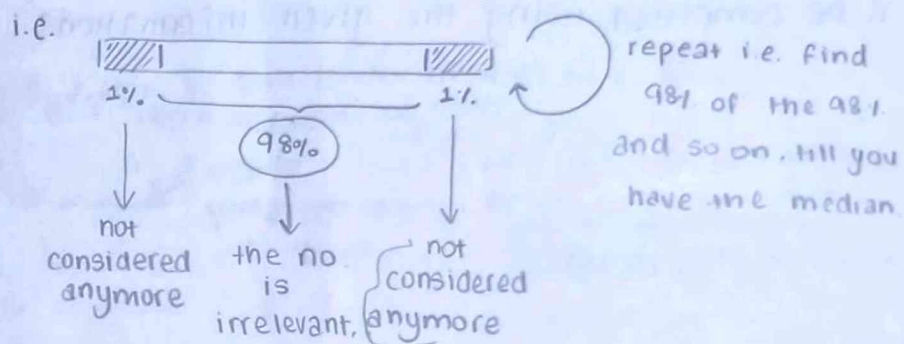
∴ $O(n)$ process

→ To find the median, we run this on an element, and if left = right, the element is at the median.

no. less
than
the element

no. greater
than
the element

ii. ∴ suppose you pick an element i.e. find median in the middle 98% of the array



Running time:

$$T(n) = T\left(\frac{98n}{100}\right) + n$$

$$= T\left(\frac{98}{100} \times \frac{98}{100} n\right) + n + \frac{98n}{100} \dots$$

⋮

$$T(1) + n + \frac{98n}{100} + \left(\frac{98}{100}\right)^k n$$

$$\hookrightarrow \frac{98 \times 98}{100 \times 100} n + \frac{98 \times 98 \times 98}{100 \times 100 \times 100} n + \dots$$

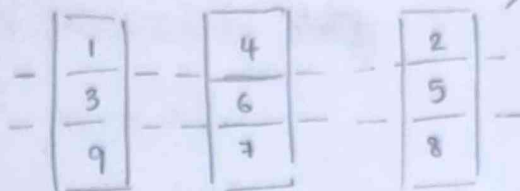
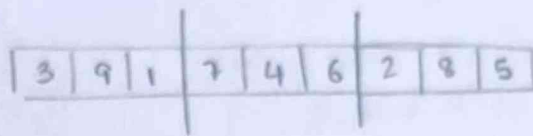
$$\therefore T(n) = \sum_{k=0}^{\infty} \left(\frac{98}{100}\right)^k n$$

$$T(n) = 50n \quad \text{i.e. } O(n) \text{ process}$$

iii. Medians of medians (MoM):

this can be used to find the median of the array we give:

First, we divide the array into multiple sub-parts, and individually sort them:

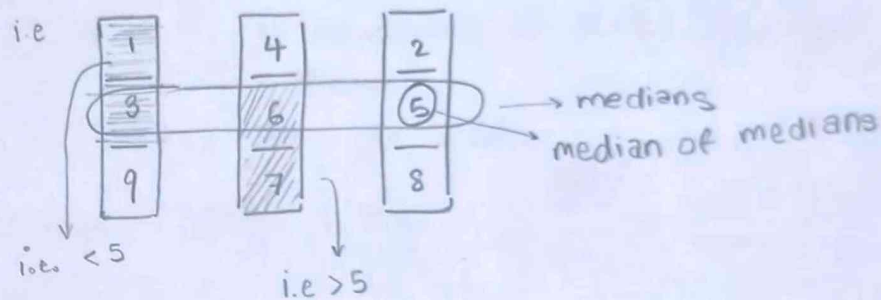


for representation only.

this step would take $2n$ comparisons for n elements.

then, we find the median of each individual sub-part, and find the median of those numbers obtained.

This method is not guaranteed to ~~give~~ give us the median itself, but it will help in eliminating numbers from the array.



None of the shaded elements can be the median, because: (will cover next class)

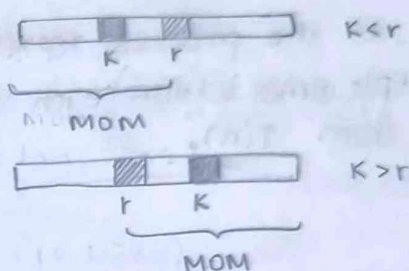
11/03/25

→ MOM: median of medians
 MOF: median of fives
 MOT: median of threes

→ median: the $[n/2]^{th}$

→ MOM(A, 1, n, K) {
 $m = \lceil n/5 \rceil$; // ceiling
 for ($i = 1$; $i \leq m$; $i++$) {
 $M[i] = MOF(A, 5i-4, 5i)$; // forms batches of 5, and sorts them.
 median = MOM(M, 1, m, $\lceil m/2 \rceil$); // called the median, may not be the actual median.
 $r = \text{partition}(A, 1, n, \text{median})$; // places the median in the same index as if A were sorted, places all the smaller elements of A to its left, and larger elements to its right.
 if ($K < r$)
 return MOM(A, 1, r-1, K);
 if ($K > r$)
 return MOM(A, r+1, n, K-r);
 return median;
}

returns the position the insertion was made at.

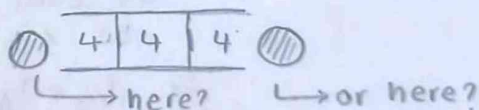


if $K = r$, element is median.

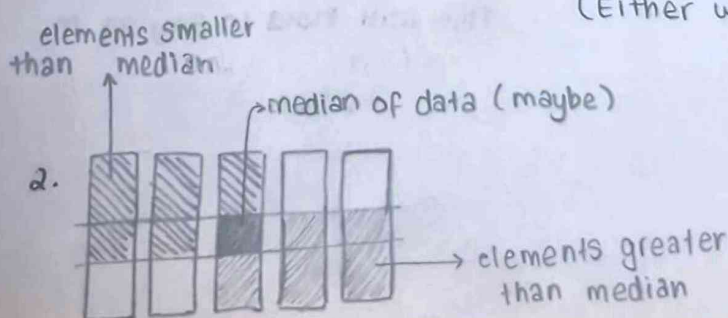
→ modifications in special cases:

1. If all elements aren't distinct, we should modify 'partition' a bit to define behaviour if the elements are equal to median

i.e. if median = 4, ~~where~~ will it go



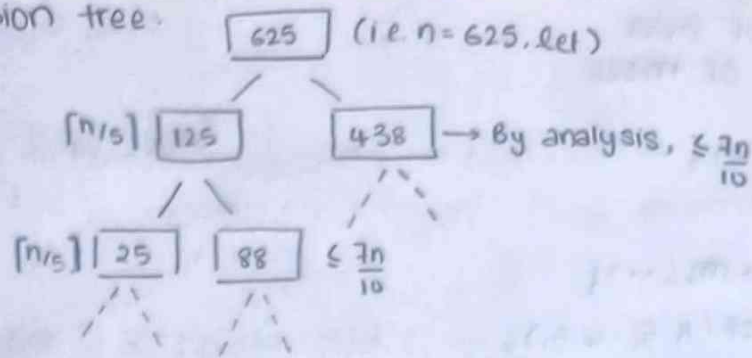
(Either works, it just has to be defined)



∴ there are $\geq \frac{3n}{10}$ elements less than $\geq \frac{3n}{10}$ elements greater than the median.

3. Code is computed in a diff. order than how it is written:

decision tree.



(will do further analysis)

4. Running time of code:

~~Why must the code be written as one block?~~

⇒ why must the code be written as one block? why can the code not be split into multiple bits?

i.e. → if this was correct, we could write T and M as two separate functions.

$T(n) \leq T(n/5) + M(n)$ is a wrong way to represent the code, as the process represented by $M(n)$ actually runs multiple times within each $T(n)$, ∴ cannot be written separately from $T(n)$.

∴ correct equation is:

$$T(n) \leq 2n + T(n/5) + 2n + T(7n/10)$$

$\downarrow$
 we need
2n swaps for n
elements

$\downarrow$
 Mof
Finding
median

$\downarrow$
 partition

$\downarrow$
 return
functions

$$\therefore T(n) \leq 4n + T(n/5) + T(7n/10)$$

Assume T is linear
i.e. n^2 and above terms
do not exist,

∴ adding up the fractions
the sum should always
be < 1

$$\text{Ex. } 1/5 + 7/10 = 9/10 < 1.$$

This will hold in any eqⁿ
 $T(cn)$.
↳ $c < 1$.

→ Assume that $T(n_0) \leq kn_0 \quad \forall n_0 < n$
↳ can be anything

Let $k=50$.

using strong induction,

$$T(n) \leq \frac{1}{5} 50n + \frac{5}{50}(7n) + 4n$$

$$\leq 10n + 35n + 4n$$

$$\leq 49n \leq 50n$$

i.e. $T(n) \leq 50n$ ∴ further proof that T is linear.

→ Why groups of 5?

5 is the smallest odd number for which this algorithm works

⇒ cannot take 1, as that does not change the problem in any way

⇒ cannot take 3, as:

$$\left\{ T(n) \leq n + T(n/3) + 2n + T(2n/3) \right\} \rightarrow \text{values obtained by same logic as in case of 5 elements.}$$

Adding the fractions,

$\frac{1}{3} + \frac{2}{3} = 1$ i.e. T is not linear in this case, as sum of fractions is ≥ 1 i.e. it is superlinear.

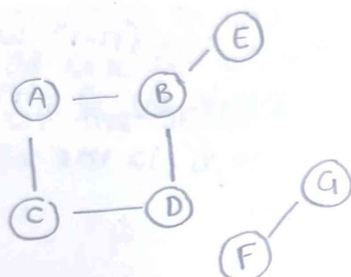
18/03/25

→ what is a graph?

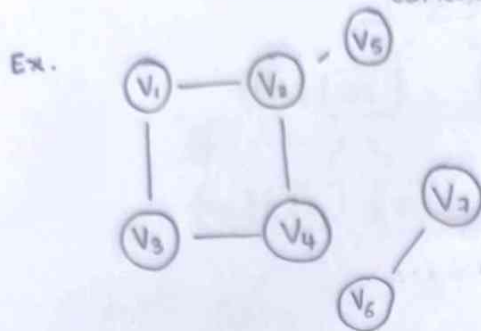
informal / casual definition

A graph is a network.

ex: computer network, social network / friendship graph, transportation network



→ formal definition. → or nodes
 → A graph $G = (V, E)$ consists of a vertex set V and edge set E such that $E \subseteq V \times V$ ← cartesian product



$$V = \{V_1, V_2, V_3, V_4, V_5, V_6, V_7\}$$

$$E = \{V_1V_2, V_1V_3, V_2V_4, V_2V_5, V_3V_4, V_4V_6, V_6V_7\}$$

i) graphs can be represented using adjacency matrix:

i.e. for an edge V_xV_y , if it exists, the (x, y) and (y, x) elements are equal to 1, and is equal to 0 if the edge doesn't exist. i.e. for the above graph:

→ (1,1)

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$A[i][j] = A[j][i]$$

→ no. of edges can be calculated by counting no. of 1s and dividing by two.

As we will not discuss self-looping and graphs with multiple edges connecting two vertices,

$A[i][i] = 0$, and any $A[i][j]$ has only one value.

Q. What is the maximum number of edges that a graph $G = (V, E)$ on n vertices (i.e. $|V| = n$) can have?

A. nC_2 edges / $\frac{n(n-1)}{2}$ edges.

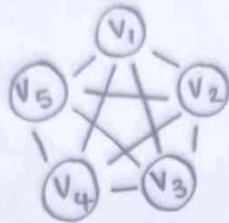
to make an edge, we pick 2 elements from n . for max. no. of edges, we have to pick 2 numbers in all possible ways, i.e. since there are nC_2 ways, there are nC_2 edges.

Each of the n vertices has $(n-1)$ ways to make an edge, and we divide by 2 since $V_iV_j = V_jV_i$ i.e. an edge from V_i to V_j is the same as an edge from V_j to V_i .

we can also count the no. of elements in the adjacency matrix:

$$\begin{array}{l} \text{total no. of elements} \rightarrow \frac{n^2 - n}{2} \rightarrow \text{remove trace elements} \\ \text{as } v_i v_j = v_j v_i \end{array}$$

Ex:



has $\frac{5 \times 4}{2} = 10$ edges

↳ complete graph. (Empty graph is a graph with no edges)

Q How many ~~are~~ different graphs are there on the vertex set

$$V = \{1, 2, 3, \dots, n\}?$$

(Two graphs are different if their adjacency matrix is different).

A. $2^{\binom{n}{2}}$ → there are $\frac{n(n-1)}{2}$ 'slots' to fill with either zero or one
(i.e. edge doesn't exist or does)

let:

$$\therefore \frac{n(n-1)}{2} = k$$

$$\begin{array}{l} \text{place 0 ones} \quad \text{place 2 ones} \\ \text{req.} = k_{C_0} + k_{C_1} + k_{C_2} + \dots + k_{C_k} \\ \text{place 1 ones} \quad \text{place k ones} \\ \therefore \text{req.} = 2^k \end{array}$$

$$\therefore \text{req.} = 2^{\binom{n}{2}}$$

also, each of the $\binom{n}{2}$ slots has two options: 0 or 1 ∴

there are $2^{\binom{n}{2}}$ ways to fill them.

→ Isomorphic graph:

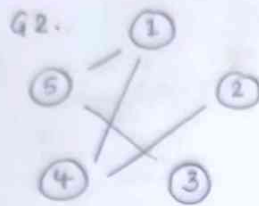
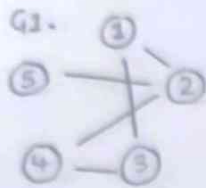
Two graphs are isomorphic to each other if the rows and columns of the adjacency matrix of one of them can be exchanged so that the adjacency matrix of the second graph is formed.

Ex. G_1 :

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

G_2 :

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$



21/03/25

→ usually, size of vertex set = n
size of edge set = m } i.e. a graph with two parameters n, m is defined like this.

→ Graph representations:

(i) Adjacency matrix :

$n \times n$ matrix = 2d array.

edge set size.

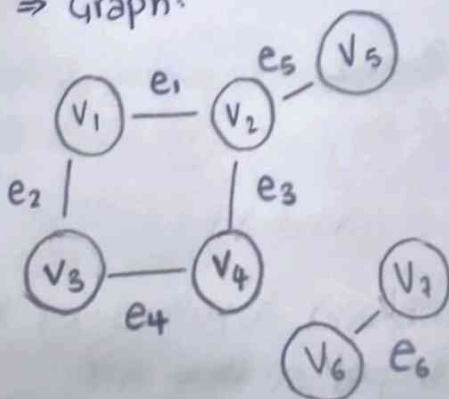
(ii) Incidence matrix = $n \times m$ matrix = 2d array.

vertex set size

(iii) Adjacency list :

n linked lists, total size $2n$.

⇒ Graph:



$n = 7$
 $m = 6$

→ checks whether two vertices are adjacent or not.
 i) Adjacency matrix:

$A[i][j] = 1$, if (i, j) is an edge

~~XXXXXXXXXX~~

	0	1	1	0	0	0	0
	1	0	0	1	1	0	0
	1	0	0	1	0	0	0
	0	1	1	0	0	0	0
	0	1	0	0	0	0	0
	0	0	0	0	0	0	1
	0	0	0	0	0	1	0

$n \times n$, i.e. 7×7

→ checks whether a vertex is incident on an edge or not
 ii) Incidence matrix:

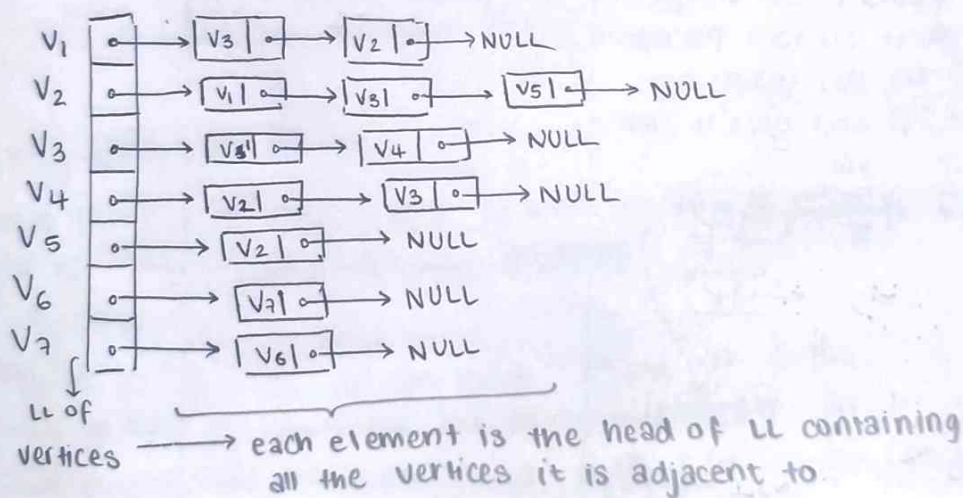
		m					
		e_1	e_2	e_3	e_4	e_5	e_6
n	v_1	1	1	0	0	0	0
	v_2	1	0	1	0	1	0
	v_3	0	1	0	1	0	0
	v_4	0	0	1	1	0	0
	v_5	0	0	0	0	1	0
	v_6	0	0	0	0	0	1
	v_7	0	0	0	0	0	1

$n \times m$ i.e. 7×6

in every column, the vertices connected by the respective edge is given 1, and all else are given 0.

iii) Adjacency list

→ we do not know sizes of individual linked lists, but the total size must be $2n$.



→ Disadvantage of incidence matrix:

1. Let $n = 10000$, each ~~column~~ has 2 '1's, and the remaining elements are all zeroes. column This is a huge waste of space.

2. The incidence matrix can also have a large number of columns.

→ Adjacency matrix vs. adjacency list:

	list	matrix
1. space for storage	$O(n+m)$ ∴ list is better for when n is large	$O(n^2)$ ↳ $n \times n$ element space req
2. To check if $(u,v) \in E$?	$\min \{ \deg(v_2), \deg(v_3) \}$ ↳ Iterate through LL of u/v and find the other.	$O(1)$ ↳ search (u,v) and see if 1 or 0
3. To find all neighbours of a vertex	$O(n)$ ↳ iterate through LL of the vertex	$O(n)$ ↳ Iterate through row/column of the vertex

✦ some terminology.

a) neighbour: a vertex v is a neighbour of u if $(u,v) \in E$.

b) degree = degree of a vertex is the number of neighbours it has. $\rightarrow 0 \leq v \leq n-1, v = \deg(v) \forall v \in V$.

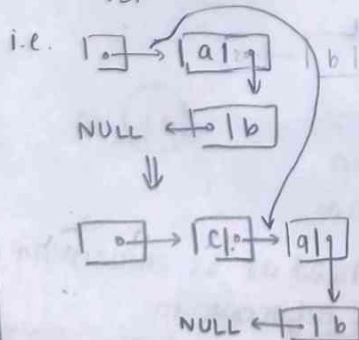
4. To find all edges of a vertex

$O(n+m)$
↳ traverse entire structure

$O(n^2)$
↳ traverse entire structure.

5. Inserting an edge

$O(1)$
↳ find LL of vertex, store current ptr out of it, insert new edge and give it that val.



$O(1)$
↳ just make

$0 \Rightarrow 1$.

6. Deleting an edge

$O(n)$
↳ traverse LL of vertex, and delete node

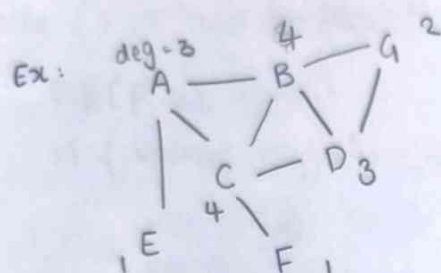
$O(1)$
↳ convert 1 to 0.

In some cases, adjacency list is better, and in some cases, adjacency matrix is better.

Overall, it is better to use adjacency list.

→ Handshaking lemma

"If ^{each} a person at a party shakes hands with ^{each of} their friends, then the total number of handshakes is equal to half the cumulative total of the number of handshakes each person makes."



∴ number of friends of person
= degree of vertex,
and friends = neighbours.

$$\begin{aligned} \therefore \text{cumulative total} &= 3 + 4 + 4 + 3 + 1 + 1 + 2 = 18 \\ \text{half} &= 18/2 = 9 \end{aligned}$$

i.e. there are 9 handshakes = no. of edges.

$$\text{i.e. } \sum_{i=1}^n \text{degree}(v_i) = 2 \times \text{no. of edges}$$

⇒ rewriting handshake lemma,

the total number of edges in a graph = $2 \times$ ~~sum of degrees~~ cumulative total of degrees of individual vertices.

⇒ Fact. The no. of vertices w/ an odd number of degree is even.

Prf: total no. of vertices w/ an even number of degree = n (let)
total no. of vertices w/ an odd number of degree = n (let)

$$\therefore \sum \text{degree}(v_i) = 2 \times |E| \text{ i.e. sum of all degrees = even}$$

$$n \times \text{even degrees} + n \times \text{odd degrees} = \text{even no.}$$

$$\therefore \text{even no.} + n \times \text{odd degrees} = \text{even no.}$$

$\therefore n \times \text{odd degrees} = \text{even no.}$

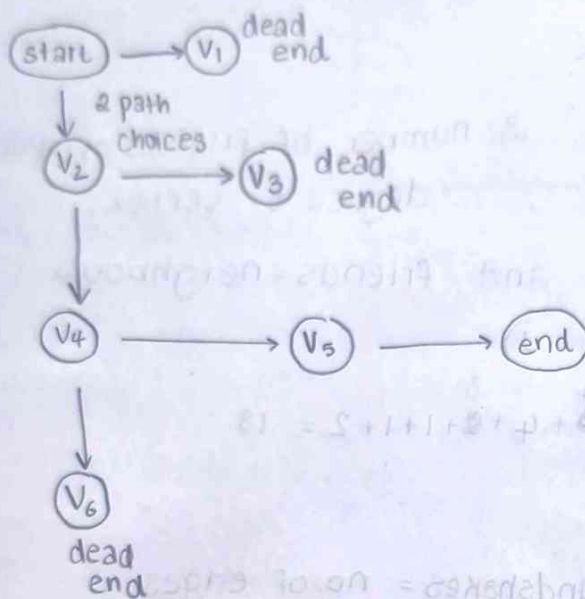
$\therefore n = \text{even no.}$

$\therefore$ Hence proved.

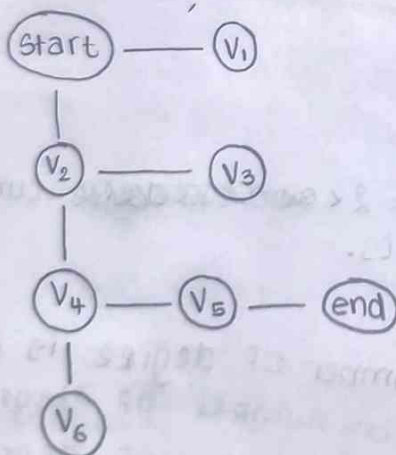
→ How to solve a maze.

→ we start from the start vertex, and go through the maze.

Any time we encounter a choice of paths, we introduce new vertices i.e. :



removing the arrows, we get a graph:

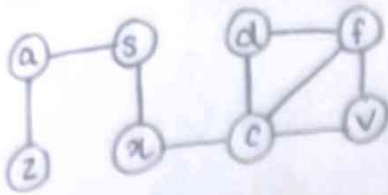


The problem is now shortest traversal from 'start' node to 'end' node.

25/05/25 → Notes not taken
28/05/25

→ Depth-First-Search (DFS)

→ employs a stack where BFS employs a queue



DFS(s) {

push (null, s) to stack S

while (S is non empty) {

pop (p, v) from S

if (visited(v) == FALSE) {

visited(v) = TRUE

parent(v) = p

for each (v, w) ∈ E,

push (v, w) to S

}

}

}

⇒ DFS on the above graph would be:

- i. looks at given node
- ii. looks at neighbours of given node
- iii. looks at neighbours of these neighbours

iv. Terminate when we have looked at all nodes in the graph.

⇒ if we do DFS(s) in above graph,

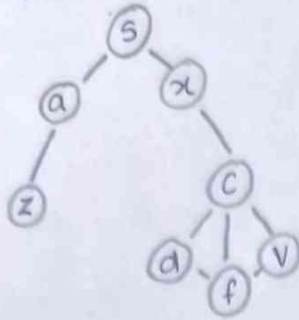
i.e.
process
is
continued
with
each node.

1. visited(s) = true

2. all neighbours of s are pushed to stack

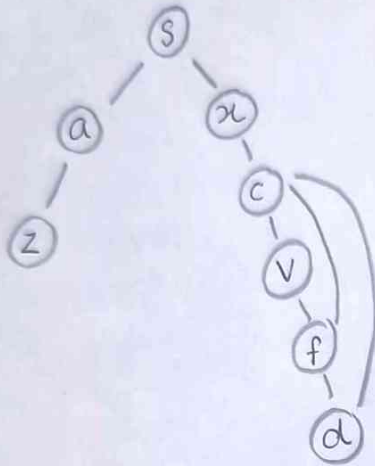
3. as each neighbour is popped,

⇒ BFS traversal :



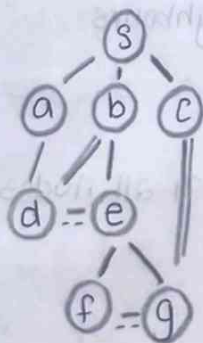
7 forward edges
2 cross edges

DFS traversal :



7 forward edges
2 back edges

Q. Construct a graph for which this is a BFS traversal. You may add more edges.



7 forward edges
2 back edges
2 cross edges.

this edge is impossible in BFS traversal.

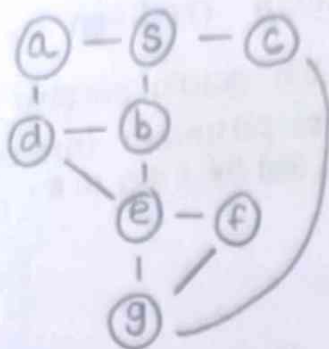
A.

→ PTO

⇒ Forward edge: edge to a parent

Cross edge: edge to a vertex in the same layer

Backward edge: any other edge.



⇒ property of BFS: Let v be a vertex in level i of a BFS traversal of a graph. Then, all its neighbours are in levels $(i-1)$, i or $(i+1)$.

↪ If there is an edge between two vertices in a BFS at levels i and j , then $|i-j| \leq 1$

⇒ The level of a vertex in the BFS traversal from s is its distance from s .

Ex. Take the above graph. paths from s to d .

$s \rightarrow c \rightarrow g \rightarrow f \rightarrow c \rightarrow d$

$s \rightarrow c \rightarrow g \rightarrow e \rightarrow d$

$s \rightarrow b \rightarrow e \rightarrow d$

$s \rightarrow b \rightarrow d$

$s \rightarrow a \rightarrow d$

} shortest paths

∴ $\text{dist}(s, d) = 2$

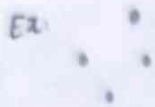
Thy. In $O(|V| + |E|)$ it is possible to compute distances of all vertices from s . Prove.

Prf: If a vertex v is at level ℓ_v in the BFS traversal, then

$\text{dist}(v, s) = \ell_v$

→ Types of graphs

1. Empty graph: A graph with no edges
2. Complete graph: A graph in which all pairs of vertices are edges
3. Path: A chain of connected vertices without repetition
4. Cycle: A loop of connected vertices without repetition
5. Tree: A graph where every pair of vertices has exactly one path connecting them
6. Bipartite graph: A graph whose vertices can be partitioned into L and R s.t. every edge has one endpoint in L, and the other in R.
7. Regular graph:
8. Planar graph:



empty graph



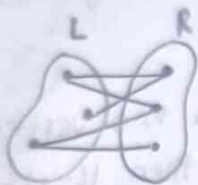
complete graph



path

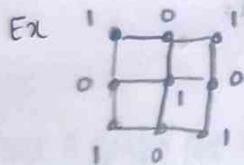


cycle



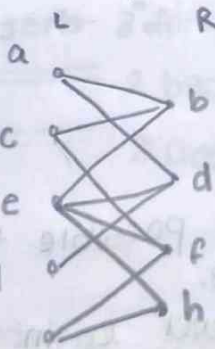
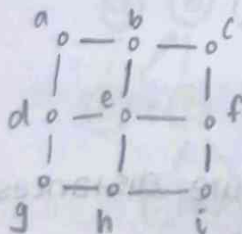
Bipartite graph

⇒ To prove a graph is bipartite, we give the ~~vertices~~ ^{vertices} one of two states:



adjacent vertices cannot have the same state,

if we can successfully assign all vertices a state, the graph is bipartite.



⇒ If we do a BFS traversal of a bipartite graph, we will not get any cross edges.

i.e. A graph is a bipartite graph iff it does not contain any cross edges.

01/01/25

Directed graphs:

→ differences b/w undirected and directed graphs.

Undirected

Directed.

→ $G = (V, E)$

→ $E \subseteq V \times V$

→ $(v, v) \in E$

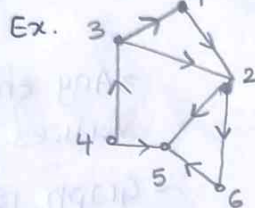
→ if $(u, v) \in E \rightarrow (v, u) \in E$

→ $A[j][i] = A[i][j]$
i.e. adjacency matrix is symmetric.

→ $G = (V, E)$

→ $E \subseteq V \times V$

→ $(v, v) \notin E$



for the above directed graph, the adjacency matrix would be:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

, clearly not symmetric.

Ex.



take this undirected graph. Convert it to a directed graph s.t. there are no cycles.

A.



is a directed graph with no cycles.

Q. Draw an undirected graph such that it is impossible to convert it into a ~~non~~ directed acyclic graph.

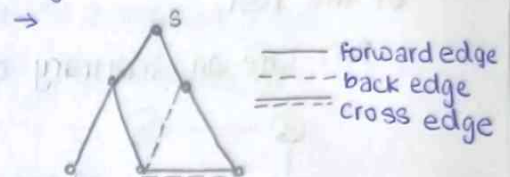
Quiz 2:

• Bipartite graphs:

→ A graph is bipartite iff its BFS traversal has no cross edges.

→ A graph is bipartite iff it has no odd-length cycles

• Cycles in BFS



→ odd cycle $\iff$ cross edge



Assume there is a cycle containing A and B. It would be through the points connected A to ion

their common root and B to their common root. This is, however, an even no. of edges. If the cycle is odd, one more edge must exist, which can only be between A and B i.e. must be a cross edge.

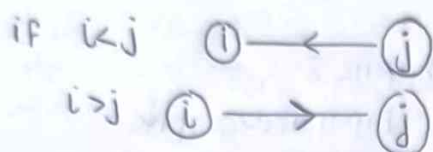
(backward ~~proof~~ proof is vice-versa)

→ even cycle $\iff$ back edge

(same proof as above, back edge adds +2 to cycle, rather than +1)

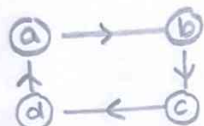
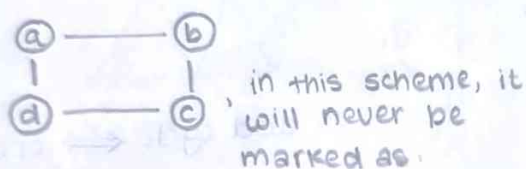
⊘. It is impossible to draw such a graph.

Take the following scheme.



by marking any graph this way, we ensure it is acyclic, as every possible cycle will have at least one edge that is in the opposite direction of the rest.

Prf: Take an arbitrary cycle.



as that would mean:

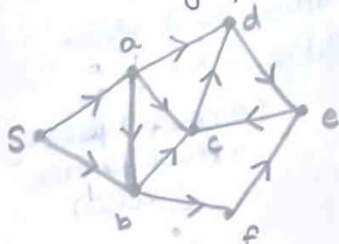
$$a > b > c > d > a \Rightarrow a > a,$$

which is not possible.

• Shortest paths

- SSSP: single source shortest paths
- Given a directed graph G and a vertex S of G (called the 'source'), compute shortest paths from S to every other vertex of the graph G .

Ex. From the graph:



here, $\text{dist}(s, e) = 3$ (shortest dist from s to e)

even cycle ~~can~~ does not imply back edges.

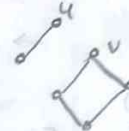
• Trees

- no odd or even cycles, $\therefore$ contain only forward edges.
- $\therefore$ every tree is a bipartite graph

• (dis)connected graphs

- A graph is disconnected iff it contains two vertices u and v such that there is no path between u and v .

Ex:



disconnected graph on 6 vertices

- Any empty graph on $n \geq 2$ vertices is disconnected.

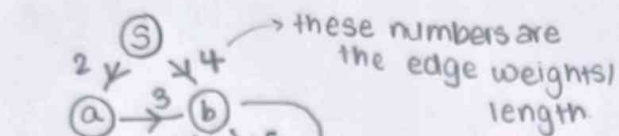
• Graph isomorphism

- Two graphs are isomorphic if one of them can be relabelled s.t. that their adjacency matrices are identical
- To prove two graphs are isomorphic, simply relabel properly
- To prove two graphs are not isomorphic, show a property that exists in one graph but not in the other

Ex: no. of points
 no. of edges
 degrees
 cycles
 etc.

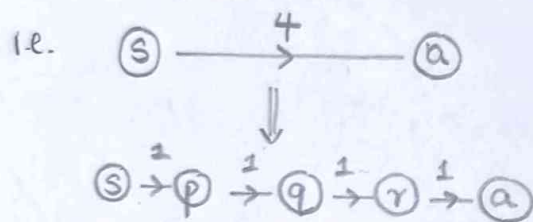
↳ quiz syllabus: graph theory

Ex.

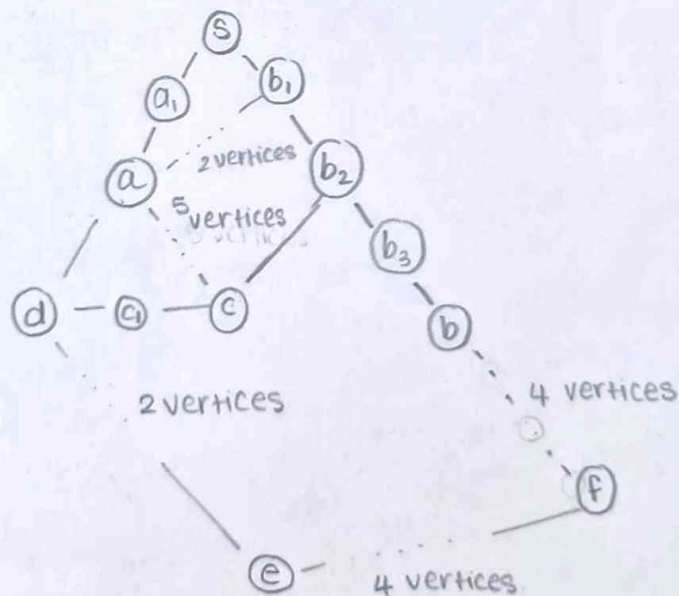


$\text{dist}(s, s) = 0$
 $\text{dist}(s, a) = 2$
 $\text{dist}(s, b) = 4$
 $\text{dist}(s, c) = 5$
 $\text{dist}(s, d) = 3$
 $\text{dist}(s, e) = 6$
 $\text{dist}(s, f) = 9$

to be able to use same algorithm as before on a weighted graph, we can simply add vertices in that path s.t. it becomes an unweighted graph:



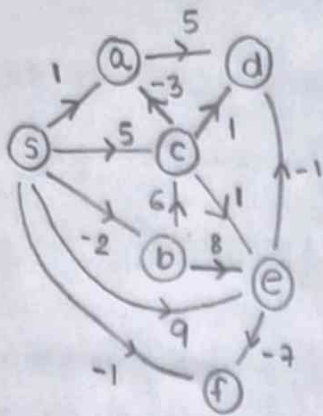
∴ above graph becomes:



04/04/25 → notes not taken

08/04/25

→ Bellman-Ford-Moore Algorithm:



we can see the final shortest distances are:

s	a	b	c	d	e	f
0	1	-2	4	4	5	-2

Now, let us traverse through graph step by step and analyse the traversal:

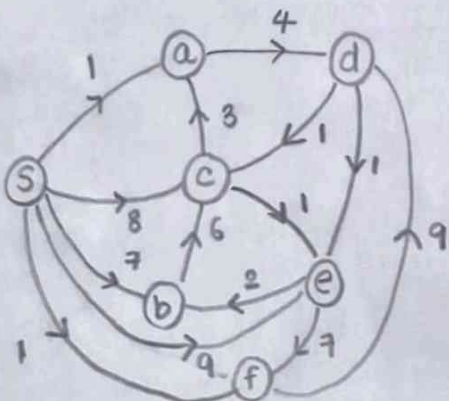
to mark that it is same as original, only for analysis.

	s	a	b	c	d	e	f	
R1 Initially	0	∞	∞	∞	∞	∞	∞	
R2	0	1	-2	5	∞	9	-1	→ traverse neighbours of s
R3	0	1	-2	4	6	6	-1	→ traverse neighbours of neighbours choose smallest value always.
R4	0	1	-2	4	5	5	-1	
R5	0	1	-2	4	4	5	-2	
R6	0	1	-2	4	4	5	-2	

→ The running time is $O(mn)$, as the algorithm has to be run ~~repeatedly~~ at least $K(v)$ times, which is $(n-1)$ times, and each time we have to check m edges, and we need 1 extra round to check for when the entire ~~array~~ list is not updated

$$\text{i.e. } O(m(n-1+1)) = O(mn).$$

Dijkstra's algorithm



Let us traverse this graph in a similar manner, with a few changes:

	s	a	b	c	d	e	f
R0	0	∞	∞	∞	∞	∞	∞
R1	0	1	7	8	8	9	1
R2	0	1	7	8	5	9	1
R3	0	1	7	6	5	6	1
R4	0	1	7	6	5	6	1
R5	0	1	7	6	5	6	1

→ since graph has only +ve edges, these are all fixed

→ continue filling in st we put shortest dist obtained

→ minimum value among the updates at each round can be fixed

→ running time of Dijkstra's algo:

In every iteration, we look at the outgoing edges from each newly updated vertex, $\therefore$ each edge is looked at exactly once.

$\therefore$ we look at $\text{out}(v_1) + \text{out}(v_2) + \dots + \text{out}(v_n) = m$
 $\hookrightarrow$ number of edges

i.e. we look at m things, over n rounds

$\therefore$ ~~running time~~ $\therefore O(m + n \log n)$

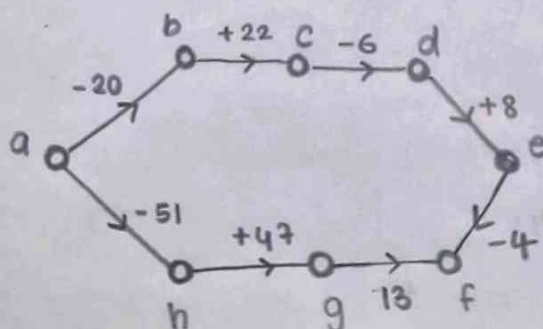
↓
to find the minimum at every round.

→ Dijkstra's algorithm cannot handle negative edge weights.

i.e. Dijkstra's algo is for non-negative edge weights.

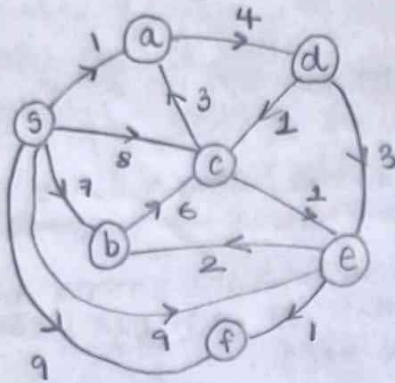
Q. ~~Q.~~ Take $\{-20, -51, -6, +8, +47, +13, -4, +22\}$, ~~and~~ and scale them up to be non-negative. Construct a graph (without -ve cycles) with 8 edges having the mentioned original edge weights s.t. Dijkstra's algorithm fails to give the correct shortest path when implemented on the updated edge weights.

A.



for $a \rightarrow f$, this graph fails for Dijkstra's algorithm.

→ 11th notes not taken
15/04/25



```
dijkstra(s){
  for (all vertices v)
    INSERT (v, dist(v)) into Q
  while (Q is nonempty)
  {
    u = extractmin from Q
    for (all edges (u, v))
      if (dist(v) > dist(u) + w((u, v)))
      {
        dist(v) = dist(u) + w((u, v))
        DECREASEKEY (v, dist(v)) in Q
      }
    }
  }
```

Q. suppose the edge weight represent the amount of time to travel on that road/street, and each vertex weight represents the amount of time spent at the junction / traffic signal.

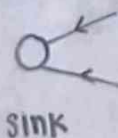
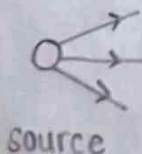
(I) How will you modify dijkstra to find a shortest path?
what lines would you add/delete/edit from dijkstra?

(A)

(II) what will you do, if you are not allowed to touch the code of dijkstra?

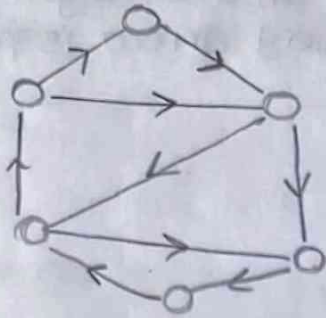
• Topological sorting

→ definition: A source vertex is a vertex of in degree 0, and a sink vertex is a vertex of out degree 0.



Q. Draw a directed graph on 6 vertices and 9 edges that has no source or sink vertex.

A-



Q. Draw a directed acyclic graph on 6 vertices and 9 edges that has no source or sink vertex.

A. This is impossible. To prove:

Q. Draw a directed graph on 6 vertices and 5 edges that has no source or sink.

A. This is impossible, every such ~~edges~~ graph has at least one source and one sink.

→ First, assume this is possible.



f cannot be traced back to any vertex i.e. either f is a sink or a is a ~~source~~ source without forming a cycle.

∴ we have a contradiction

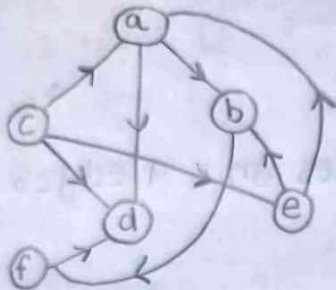
∴ The statement is impossible ∴ Hence proved

→ ∴ Every directed acyclic graph has at least one source and one sink.

→ Topological Sort :

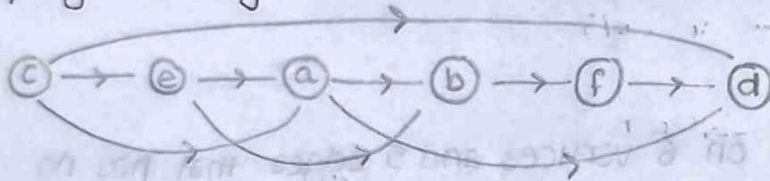
Arrange vertices of a directed acyclic graph on a straight line from left to right s.t. the direction of every arrow is from left towards right.

Ex



graph G

topological sorting :

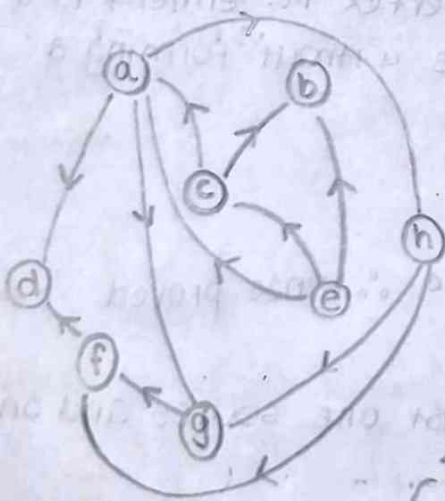


→ If a graph has a topological sorting, then it is a directed acyclic graph, and given a directed acyclic graph it is possible to topologically sort it in $O(m+n)$ time.

x

17/04/25

Ex.



Q1. Draw a topological sorting of the given graph.

(leftmost vertex must be source and rightmost must be sink)

A.

source vertex

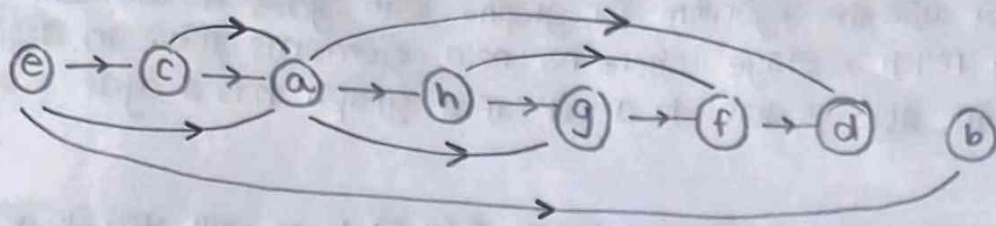


main sort

After e + b are removed, this is source vertex

↳ sink vertex also placed here

doing toposort:

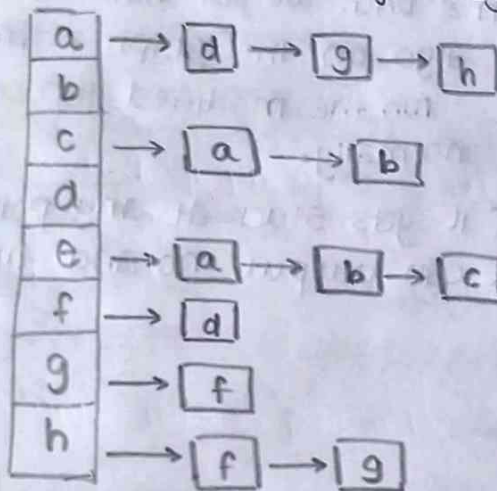


→ There can be multiple ways to topologically sort a graph.

→ Removing a vertex and all the relevant ~~graph~~ edges from a DAG does not change the fact that the graph is a DAG.

→ Algorithm.

take the above graph in adjacency list form:



Let us construct:

out degree	
a	3
b	0
c	2
d	0
e	3
f	1
g	1
h	2

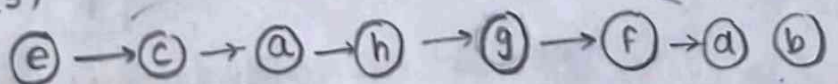
indegree	
a	2
b	2
c	1
d	2
e	0
f	2
g	2
h	1

→ not removed yet as it also has out-degree 0 → remove vert. w/ higher out degree

Now, we find vertex w/ in-degree 0, ~~delete~~ ~~mark~~ ~~in~~ array, write it down, and reduce

~~degrees~~ degrees of all the elements this vertex was connected to

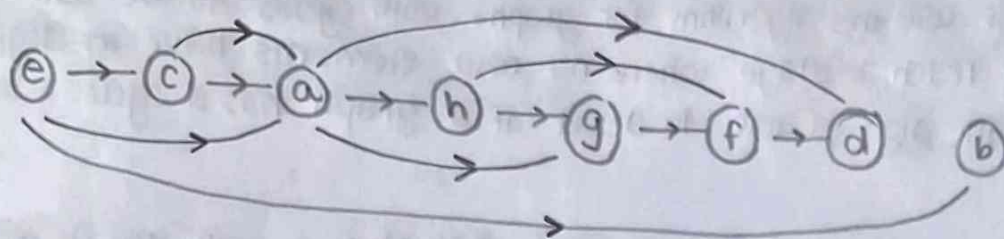
i.e.



make remaining connections

This is done in $O(|V| + |E|)$ time.

doing toposort:

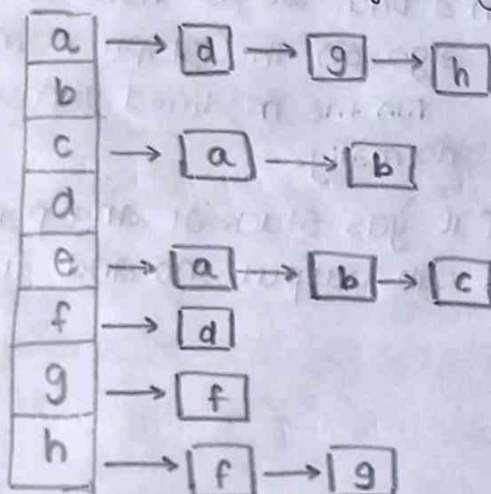


→ There can be multiple ways to topologically sort a graph.

→ Removing a vertex and all the relevant ~~graphs~~ edges from a DAG does not change the fact that the graph is a DAG.

→ Algorithm.

take the above graph in adjacency list form:



let us construct:

out degree	
a	3
b	0
c	2
d	0
e	3
f	1
g	1
h	2

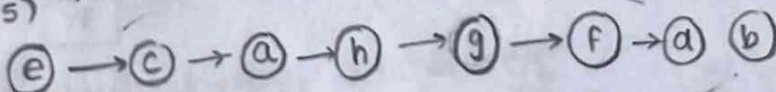
indegree	
a	2
b	2
c	1
d	2
e	0
f	2
g	2
h	1

not removed yet as it also has out-degree 0 → remove vert. w/ higher out degree

Now, we find vertex w/ in-degree 0, ~~delete~~ ^{mark it} in ~~array~~ array, write it down, and reduce

~~array~~ degrees of all the elements this vertex was connected to

ie.



make remaining connections

This is done in $O(|V| + |E|)$ time.

→ we cannot use this algorithm for graphs with cycles as we will eventually reach a stage where no new elements have in degree 0. Thus, the algo is able to detect if a graph has a cycle.

Q. Suppose you are given a directed graph $G(V, E)$. How will you check if it is a DAG in $O(V+E)$ time?

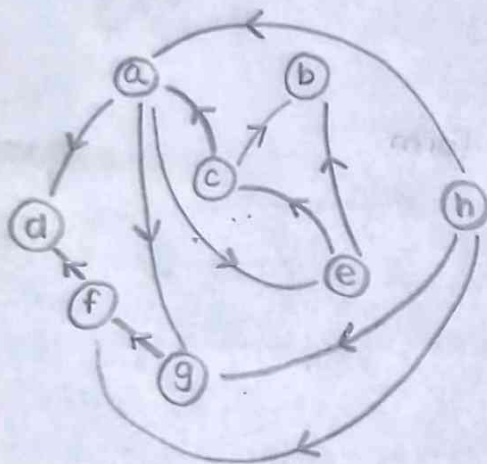
If the graph is a DAG output toposort

If it is not a DAG output a cycle.

(Hint: use the out degree)

A.

Example graph (contains cycle):

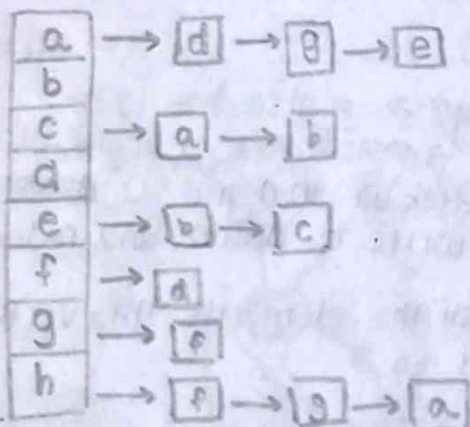


If we want to find the toposort in a DAG, we just have to run the algo on the graph normally / run the modified toposort algo normally.

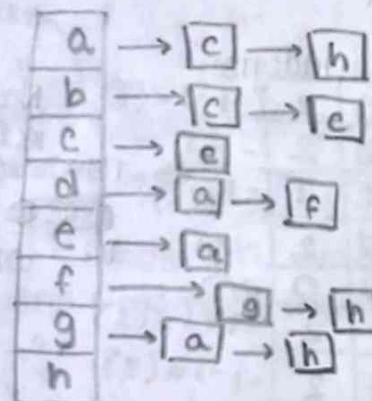
If it gets stuck at any point, we can output no and find cycles.

How to find cycle:

adj list:



reversed adj list (traverse adj list and if $a \rightarrow b$, mark $b \rightarrow a$ here)

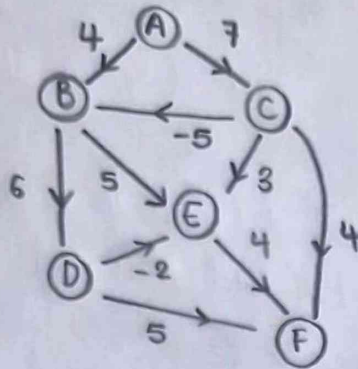


now we can traverse through the vertices of the reversed adj list that has non zero in degree.

Q. How will you use topological sorting to find shortest path in a DAG in $O(|V| + |E|)$ time.

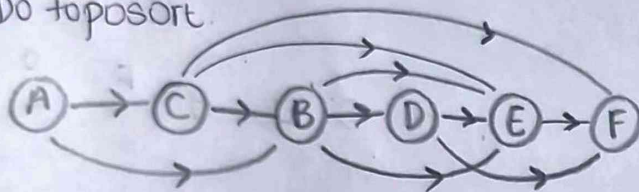
A. Run the comparison algo (dijkstra's) on the topo sort of the graph.

~~Ex:~~ Ex:

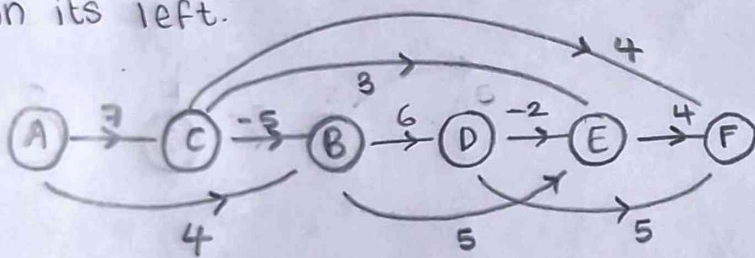


To find shortest path:

a) Do toposort.



b) let source = A. let us fill numbers. since there are no back edges in a topological sort, you can only reach a vertex ~~once~~ via the vertices on its left.



dist(F) = 10

dist(A) = 0

dist(C) = 7

dist(D) = 8

dist(E) = 6

dist(B) = 2

— X —

→ 22nd notes not there, refer external notes on hashing.